

**TRƯỜNG ĐẠI HỌC YERSIN ĐÀ LẠT
KHOA CÔNG NGHỆ THÔNG TIN**



YERSIN UNIVERSITY

**BÁO CÁO MÔN HỌC
LẬP TRÌNH WEB 1**

**WEBSITE THƯƠNG MẠI ĐIỆN TỬ KINH
DOANH SẢN PHẨM TRUYỆN ANIME**

**GVHD : Nguyễn Đức Tấn
SVTH : K' Sơn
Mã số SV : 2301010074
Khóa học : 2024 - 2025**

Đà Lạt, tháng 06 - 2025

This image shows a full page of white paper with horizontal dashed lines, typical of primary-ruled notebook paper. The lines are evenly spaced and run across the entire width of the page. There are no margins, text, or other markings present.

Ký và ghi rõ họ tên

MỤC LỤC

LỜI NÓI ĐẦU	6
CHƯƠNG 1 :TÌM HIỂU VỀ LẬP TRÌNH WEB MVC (CƠ SỞ LÝ THUYẾT)	7
1.1 Tổng quan về ASP.NET Core:.....	7
1.1.1 Giới thiệu về ASP.NET Core:.....	7
1.1.2 Tìm hiểu về mô hình lập trình web MVC của ASP.NET Core:	7
1.2 Nguyên lý hoạt động ASP.NET MVC CORE:.....	9
1.2.1 Khái niệm:.....	9
1.2.2 Ví dụ minh họa: Xây dựng ứng dụng quản lý sản phẩm đơn giản theo mô hình MVC	10
1.3 Đặc Điểm:	12
1.3.1 Tại sao phải sử dụng ASP.NET Core? :	12
1.4 Công nghệ triển khai ASP.NET Core:.....	15
1.4.1 Môi trường triển khai:.....	15
1.4.2 Các hình thức triển khai:.....	15
1.4.3 Các nền tảng đám mây hỗ trợ:	16
1.4.4 Công cụ hỗ trợ triển khai:	16
CHƯƠNG 2 :XÂY DỰNG ỨNG DỤNG THỰC TẾ	17
2.1 Phát biểu bài toán ứng dụng:.....	17
2.1.1 Giới thiệu dự án:	17
2.1.2 Vai trò của dự án:.....	17
2.1.3 Ý nghĩa của dự án:	18
2.2 Phân Tích yêu cầu của ứng dụng:	18
2.2.1 Mô tả chức năng ứng dụng:	18
2.2.2 Mô tả yêu cầu phi chức năng của ứng dụng:	22
2.2.3 Sơ đồ UseCase:	24
2.3 Thiết kế cơ sở dữ liệu:.....	27
2.3.1 Thiết kế logic ERD:	27
2.3.2 Mô hình vật lý	29
2.3.3 Mô tả chi tiết các bảng	30
2.4 Thiết kế giao diện:.....	33

2.4.1 Trang chủ:	33
2.4.2 Trang danh mục (Thẻ loại / Loại truyện/ Whishlist)	34
2.4.3 Trang chi tiết sản phẩm.....	34
2.4.4 Giỏ hàng.....	35
2.4.5 Profile.....	36
2.4.6 Trang Admin	36
2.5 Thiết kế các thành phần MVC:	37
2.5.1 Model :	37
2.5.2 View	42
2.5.3 Controller	44
2.6 Triển khai cài đặt.....	50
2.6.1 Cấu hình máy phát triển.....	50
2.6.2 Môi trường phát triển.....	50
2.6.3 Cài đặt	51
CHƯƠNG 3 KẾT QUẢ CHƯƠNG TRÌNH	52
3.1 User	52
3.1.1 Trang chủ	52
3.1.2 Chi tiết sản phẩm	53
3.1.3 Danh mục/Danh sách yêu thích	54
3.1.4 Đăng nhập/Đăng ký	55
3.1.5 Giỏ hàng.....	56
3.1.6 Thanh toán.....	57
3.1.7 Lịch sử đơn hàng.....	58
3.1.8 Liên Hệ.....	59
3.1.9 Profile.....	60
3.1.10 Đổi mật khẩu/Phản hồi.....	61
3.2 Admin.....	62
3.2.1 Dashboard	62
3.2.2 Quản Lý User	62
3.2.3 Quản lý Danh mục	62
3.2.4 Quản lý sản phẩm.....	63
3.2.5 Quản Lý đơn hàng.....	63
3.2.6 Quản lý liên hệ	64

3.3 Tổng quan	64
CHƯƠNG 4 : KIẾT LUẬN	65
4.1 Tổng kết kiến thức đạt được	65
4.2 Điểm tồn tại.....	65
4.3 Hướng phát triển dự án	66
4.4 Tài liệu tham khảo:	66

LỜI NÓI ĐẦU

Trong kỷ nguyên số hiện nay, công nghệ thông tin đã trở thành một phần không thể thiếu, định hình mọi khía cạnh của đời sống và kinh doanh. Đặc biệt, sự bùng nổ của AI và các nền tảng kỹ thuật số đã thúc đẩy mạnh mẽ sự phát triển của thương mại điện tử, mở ra vô vàn cơ hội và thách thức cho các doanh nghiệp lẫn người dùng cá nhân. Các ứng dụng web quản lý và bán hàng trực tuyến không chỉ đơn thuần là công cụ hỗ trợ mà đã trở thành yếu tố then chốt, quyết định khả năng cạnh tranh và tiếp cận thị trường của một tổ chức.

Trước những biến chuyển sâu sắc đó, việc nghiên cứu, phát triển và ứng dụng các giải pháp công nghệ mới nhằm đáp ứng nhu cầu thị trường là hết sức cần thiết. Đặc biệt, đối với sinh viên công nghệ thông tin, việc thực hiện các đồ án mang tính ứng dụng cao không chỉ giúp củng cố kiến thức lý thuyết, mà còn rèn luyện kỹ năng lập trình, tư duy hệ thống và khả năng giải quyết vấn đề trong môi trường thực tế.

Đồ án này là kết quả của quá trình tìm hiểu, nghiên cứu và triển khai một ứng dụng web cụ thể, nhằm góp phần vào xu thế chung của ngành công nghệ thông tin. Báo cáo này sẽ trình bày chi tiết về quá trình xây dựng một ứng dụng web hoàn chỉnh, từ việc phân tích yêu cầu, lựa chọn công nghệ, đến thiết kế kiến trúc, triển khai và đánh giá kết quả. Qua đó, mình mong muốn thể hiện khả năng ứng dụng lý thuyết vào thực tiễn, giải quyết một bài toán cụ thể trong lĩnh vực thương mại điện tử.

Em xin được bày tỏ lòng biết ơn sâu sắc đến thầy Nguyễn Đức Tấn, người đã luôn tận tình hướng dẫn và hỗ trợ em trong suốt quá trình thực hiện đồ án này. Những kiến thức và kinh nghiệm quý báu cùng với sự định hướng của thầy là nền tảng quan trọng giúp em hoàn thành dự án.

CHƯƠNG 1 :TÌM HIỂU VỀ LẬP TRÌNH WEB MVC (CƠ SỞ LÝ THUYẾT)

1.1 Tổng quan về ASP.NET Core:

1.1.1 Giới thiệu về ASP.NET Core:

Đầu năm 2002, Microsoft giới thiệu một kỹ thuật lập trình Web khá mới mẻ với tên gọi ban đầu là ASP+, sau này chính thức là **ASP.NET**. Với ASP.NET, bạn không cần phải biết các thẻ HTML hay thiết kế web mà vẫn được hỗ trợ mạnh mẽ lập trình hướng đối tượng trong quá trình xây dựng và phát triển ứng dụng Web. ASP.NET là kỹ thuật lập trình và phát triển ứng dụng web ở phía máy chủ (Server-side) dựa trên nền tảng của Microsoft .NET Framework.

Hầu hết những người mới làm quen với lập trình web đều bắt đầu với các kỹ thuật phía máy khách (Client-side) như HTML, JavaScript, CSS (Cascading Style Sheets). Khi trình duyệt web yêu cầu một trang web (sử dụng kỹ thuật Client-side), máy chủ web sẽ tìm trang đó và gửi về cho máy khách. Máy khách nhận kết quả từ máy chủ và hiển thị lên màn hình.

ASP.NET Core sử dụng kỹ thuật lập trình phía máy chủ hoàn toàn khác. Mã lệnh ở phía máy chủ (ví dụ: mã lệnh trong trang ASP.NET) sẽ được biên dịch và thực thi tại Web Server. Sau khi được Server đọc, biên dịch và thực thi, kết quả sẽ tự động được chuyển sang HTML/JavaScript/CSS và trả về cho Client. Tất cả các xử lý lệnh ASP.NET Core đều được thực hiện tại Server, do đó được gọi là kỹ thuật lập trình phía máy chủ.

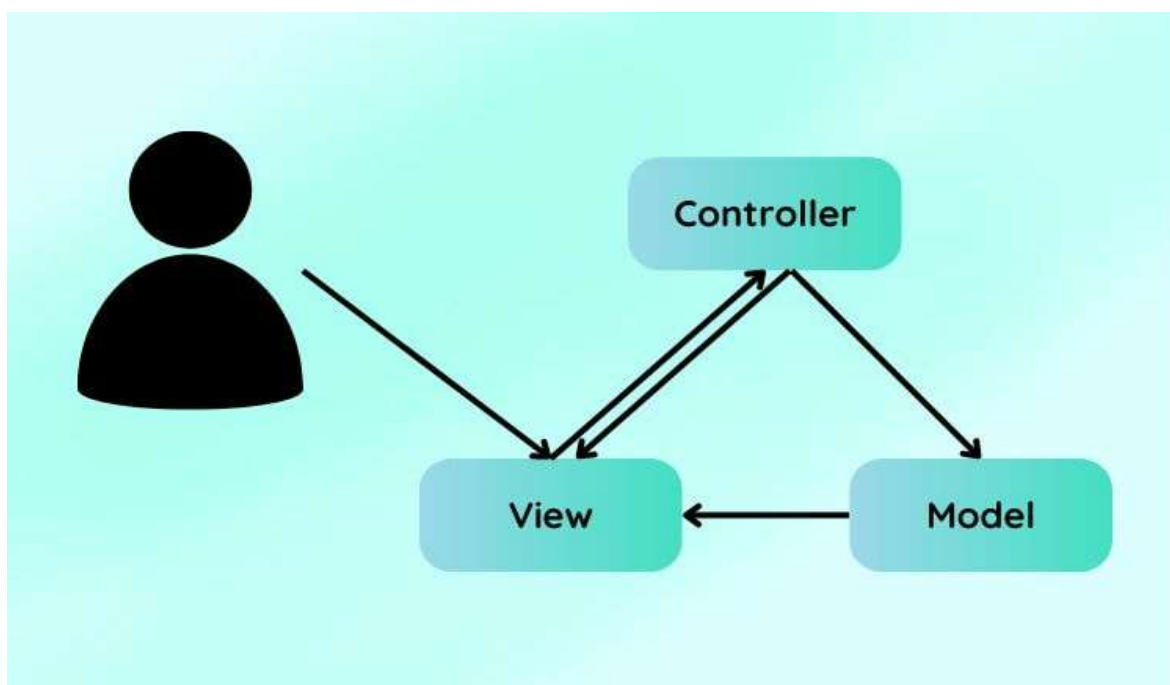
1.1.2 Tìm hiểu về mô hình lập trình web MVC của ASP.NET Core:

Mô hình **MVC** (viết tắt của Model - View - Controller) là một kiến trúc phần mềm hay mô hình thiết kế được sử dụng trong kỹ thuật phần mềm (đặc biệt đối với phát triển ứng dụng web). Nó giúp tổ chức ứng dụng (phân chia mã nguồn) thành ba phần khác nhau: **Model, View và Controller**. Mỗi thành phần có một nhiệm vụ riêng biệt và độc lập với các thành phần khác.

Model: Chứa tất cả các nghiệp vụ logic, phương thức xử lý, truy xuất cơ sở dữ liệu (CSDL), đối tượng mô tả dữ liệu như các Class, hàm xử lý... Model có nhiệm vụ cung cấp dữ liệu và lưu dữ liệu vào các kho chứa. Tất cả các nghiệp vụ logic được thực thi ở Model. Dữ liệu từ người dùng sẽ thông qua View để kiểm tra ở Model trước khi lưu vào cơ sở dữ liệu. Việc truy xuất, xác nhận và lưu dữ liệu là một phần của Model.

View: Hiển thị thông tin cho người dùng của ứng dụng và có nhiệm vụ nhận dữ liệu đầu vào từ người dùng, gửi yêu cầu người dùng đến bộ điều khiển (Controller), sau đó nhận phản hồi từ bộ điều khiển và hiển thị kết quả cho người dùng. Các trang HTML, các công cụ tạo giao diện (như Razor Pages trong ASP.NET Core) và các tệp nguồn liên quan là một phần của View.

Controller: Là tầng trung gian giữa Model và View. Controller có nhiệm vụ nhận các yêu cầu từ người dùng (phía máy khách). Một yêu cầu được nhận từ máy khách sẽ được xử lý bởi một chức năng logic thích hợp từ thành phần Model và sau đó sinh ra kết quả cho người dùng, được thành phần View hiển thị.

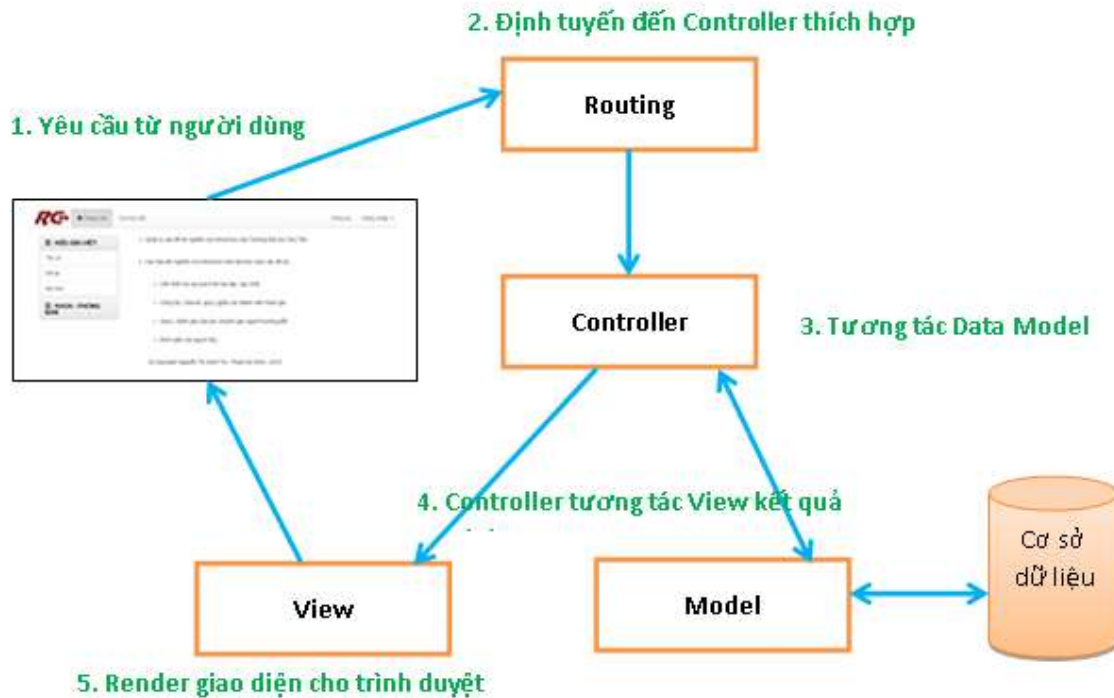


Hình 1.1: Mô hình MVC

1.2 Nguyên lý hoạt động ASP.NET MVC CORE:

1.2.1 Khái niệm:

- **Yêu cầu từ trình duyệt đến Web Server:** Khi một yêu cầu (request) được gửi từ trình duyệt web đến Web Server (ví dụ: Kestrel, IIS, Nginx), yêu cầu đó sẽ được chuyển đến Middleware Pipeline của ASP.NET Core. Trong pipeline này, cơ chế Routing Middleware sẽ đóng vai trò quan trọng trong việc xác định lộ trình phù hợp.
- **Routing và chọn Controller:** Routing Middleware có nhiệm vụ phân tích URL của yêu cầu và quyết định yêu cầu này sẽ được xử lý bởi **Controller** và **Action Method** cụ thể nào. Dựa vào cấu hình định tuyến (routes) đã được thiết lập, hệ thống sẽ ánh xạ URL đến đúng Controller và Action. Sau khi xác định được, một instance của Controller sẽ được tạo ra.
- **Thực thi Action Method:** Controller sau đó sẽ xác định **Action Method** cụ thể để xử lý yêu cầu và tiến hành thực thi Action Method đó. Trong quá trình này, Action Method có thể tương tác với các **Model Class** để truy xuất dữ liệu từ cơ sở dữ liệu hoặc thực thi các logic nghiệp vụ cần thiết.
- **Trả về Action Result:** Sau khi hoàn tất xử lý, Action Method sẽ trả về một **Action Result**. ASP.NET Core MVC cung cấp nhiều loại Action Result khác nhau (ví dụ: **ViewResult**, **JsonResult**, **ContentResult**, **RedirectResult**, v.v.). Trong đó, **ViewResult** là một loại Action Result đặc biệt. **ViewResult** có nhiệm vụ làm việc với một **View** cụ thể để tạo ra mã HTML.
- **Hiển thị View với View Engine:** View Engine (ví dụ: **Razor View Engine**) là thành phần thực hiện việc hiển thị một View. View Engine sẽ nhận dữ liệu từ Model (nếu có) và sử dụng cú pháp của mình để kết xuất thành mã HTML. Mã HTML này sau đó sẽ được trả về cho trình duyệt web và hiển thị cho người dùng, hoàn tất quá trình xử lý yêu cầu.



Hình 1.2: Mô tả Nguyên Lý hoạt động MVC

1.2.2 Ví dụ minh họa: Xây dựng ứng dụng quản lý sản phẩm đơn giản theo mô hình MVC

Model:

```

namespace Atsumaru.Models
{
    public class Product { public int Id { get; set; }
    {
        public string Name { get; set; }
        public decimal Price { get; set; }
        public string Description { get; set; }
    }
}

```

Controller:

```

namespace Atsumaru.Controllers
{
    private static List<Product> _products = new List<Product>
    {

```

```

new Product { Id = 1, Name = "Truyện One Piece Tập 1", Price = 30.000M,
Description = "Khởi đầu hành trình của Luffy." },
new Product { Id = 2, Name = "Mô hình Naruto Sage Mode", Price =
500.000M, Description = "Mô hình cao cấp." },
new Product { Id = 3, Name = "Light Novel Sword Art Online Tập 1", Price
= 95.000M, Description = "Thế giới ảo đầy kịch tính." }

```

```
};
```

```

public IActionResult Index()
{
    return View(_products);
}

```

View:

```
@model List<Atsumaru.Models.Product>
```

```
<h1>Danh sách sản phẩm Anime</h1>
```

```
<table class="table">
```

```
<thead>
```

```
<tr>
```

```
<th>Tên sản phẩm</th>
```

```
<th>Giá</th>
```

```
<th>Mô tả</th> <th></th>
```

```
</tr>
```

```
</thead>
```

```
<tbody>
```

```
@foreach (var item in Model)
```

```
{
```

```
<tr>
```

```
<td>@Html.DisplayFor(modelItem=>item.Name)</td>
```

```
<td>@Html.DisplayFor(modelItem=>tem.Price)VNĐ</td>
```

```
<td>@Html.DisplayFor(modelItem => item.Description)</td>
```

```
</tr>
```

```
}  
</tbody>  
</table>
```

1.3 Đặc Điểm:

1.3.1 Tại sao phải sử dụng ASP.NET Core? :

Yêu cầu về xây dựng các ứng dụng thương mại điện tử ngày càng phát triển và nâng cao. Khi đó, các công nghệ cũ như ASP không còn đáp ứng được yêu cầu đặt ra. ASP được thiết kế riêng biệt và nằm ở tầng phía trên hệ điều hành Windows và Internet Information Service, do đó các công dụng của nó hết sức rời rạc và giới hạn. **ASP.NET Core** ra đời mang đến một phương pháp phát triển hoàn toàn mới, khác hẳn so với ASP trước kia và đáp ứng được các yêu cầu khắt khe của ứng dụng hiện đại. Ưu điểm và khuyết điểm của ASP.NET:

Ưu điểm:

- Hỗ trợ đa ngôn ngữ lập trình: Trong khi ASP chỉ sử dụng VBScript và JavaScript, **ASP.NET Core** cho phép bạn viết code bằng nhiều ngôn ngữ mạnh mẽ như **C#**, **F#**, và **Visual Basic .NET**.
- Phong cách lập trình Code-behind hiện đại: **ASP.NET Core** thúc đẩy phong cách lập trình **Code-behind**, giúp tách biệt rõ ràng giữa logic nghiệp vụ (code) và giao diện người dùng (markup). Điều này giúp mã nguồn dễ đọc, dễ quản lý và bảo trì hơn rất nhiều.
- Hỗ trợ kiểm tra dữ liệu đầu vào mạnh mẽ: Thay vì phải viết mã thủ công để kiểm tra dữ liệu nhập từ người dùng, **ASP.NET Core** hỗ trợ các **Validation Controls** (trong ASP.NET truyền thống) và **Data Annotations** (trong ASP.NET Core) giúp việc kiểm tra dữ liệu trở nên đơn giản và hiệu quả, giảm thiểu lượng code phải viết.
- Phát triển đa nền tảng và thiết bị: **ASP.NET Core** hỗ trợ phát triển các ứng dụng web có thể truy cập trên nhiều thiết bị khác nhau, từ máy tính để bàn đến các thiết bị di động như PocketPC, Smartphone, v.v., nhờ vào khả năng đa nền tảng của nó.

- Hỗ trợ Web Server Controls (trong ASP.NET truyền thống) và Tag Helpers (trong ASP.NET Core): Cung cấp nhiều control sẵn có giúp tăng tốc độ phát triển giao diện.
- Cá nhân hóa giao diện: Cho phép người dùng thiết lập giao diện trang web theo sở thích cá nhân sử dụng các tính năng như **Theme**, **Profile**, và **WebPart** (trong ASP.NET truyền thống) hoặc các cơ chế tùy chỉnh giao diện linh hoạt trong **ASP.NET Core**.
- Tăng cường tính năng bảo mật: **ASP.NET Core** được thiết kế với các tính năng bảo mật mạnh mẽ, giúp bảo vệ ứng dụng khỏi các lỗ hổng phổ biến.
- Hỗ trợ kỹ thuật truy cập dữ liệu LINQ: Tích hợp mạnh mẽ với **LINQ (Language Integrated Query)**, giúp truy vấn dữ liệu một cách trực quan và hiệu quả.
- Tận dụng .NET Framework (nay là .NET): **ASP.NET Core** kế thừa và tận dụng mạnh mẽ bộ thư viện phong phú và đa dạng của nền tảng **.NET**, hỗ trợ làm việc với XML, Web Service, truy cập cơ sở dữ liệu qua ADO.NET (hoặc Entity Framework Core), v.v.
- Kiến trúc lập trình quen thuộc: Kiến trúc lập trình của **ASP.NET Core** có nhiều điểm tương đồng với việc phát triển ứng dụng trên Windows, giúp lập trình viên dễ dàng tiếp cận và làm quen.
- Quản lý ứng dụng ở mức toàn cục: Hỗ trợ quản lý ứng dụng ở mức toàn cục với các tính năng như Global.asax (trong ASP.NET truyền thống) hoặc middleware trong ASP.NET Core, quản lý session trên nhiều Server, và không nhất thiết cần Cookies.

Nhược điểm:

- Phức tạp hơn cho dự án nhỏ: Đối với các dự án web quy mô rất nhỏ và đơn giản, việc áp dụng mô hình MVC có thể gây cảm giác cồng kềnh và tốn thời gian hơn trong quá trình phát triển ban đầu, do cần cấu hình và thiết lập nhiều thành phần.

- Thời gian trung chuyển dữ liệu giữa các thành phần: Mặc dù là nhược điểm nhỏ, việc phân tách các thành phần Model, View, Controller có thể đôi khi làm tăng thời gian trung chuyển dữ liệu giữa chúng so với các kiến trúc monolithic đơn giản hơn. Tuy nhiên, lợi ích về khả năng mở rộng và bảo trì thường vượt trội hơn.

Bảng 1.1 Sự khác biệt mô hình lập trình Web ASP.NET MVC, ASP.NET Webform và ASP.NET Core:

Các tính năng	ASP.NET WebForm	ASP.NET MVC	ASP.NET Core MVC
Kiến trúc chương trình	Kiến trúc mô hình WebForm → Business → Database	Phân chia chương trình thành: Models, Views, Controllers	Giống ASP.NET MVC, nhưng tối ưu hơn, hỗ trợ kiến trúc Modular, Middleware, Dependency Injection
Cú pháp chương trình	Dùng cú pháp WebForm, sự kiện & controls do server quản lý	Sự kiện do controllers điều khiển, controls không do server quản lý	Cú pháp linh hoạt hơn, hỗ trợ Razor Pages, Tag Helpers, dễ tích hợp JavaScript/SPA
Truy cập dữ liệu	Dùng các công nghệ truyền thống như ADO.NET	Chủ yếu dùng LINQ, Entity Framework, SQL Class	Ưu tiên Entity Framework Core, LINQ, hỗ trợ tốt cho NoSQL, DbContext được tối ưu hóa
Debug	Debug toàn bộ (data layer, UI, controls...)	Dùng unit test để kiểm tra controllers dễ dàng hơn	Tích hợp sâu với unit test/xUnit, hỗ trợ tốt test DI, test middleware, kiểm thử REST API
Tốc độ phân tải	Chậm nếu nhiều controls do ViewState lớn	Nhanh hơn vì không dùng ViewState	Nhanh và tối ưu hơn nhờ không dùng ViewState, caching tốt, hỗ trợ async/await
Tương tác với JavaScript	Khó khăn vì controls bị server kiểm soát	Dễ hơn vì không có server-side controls	Tối ưu nhất: hỗ trợ full SPA frameworks như React, Angular, Vue, tích hợp SignalR dễ dàng

URL Address	Dạng <filename>.aspx?&<các tham số>	Cấu trúc rõ ràng: Controllers/Action/ID	Cấu trúc rõ hơn, hỗ trợ routing tùy chỉnh, RESTful API, endpoint routing mới
-------------	---	--	---

1.4 Công nghệ triển khai ASP.NET Core:

ASP.NET Core là một nền tảng phát triển ứng dụng web hiện đại, đa nền tảng và mã nguồn mở, được Microsoft phát triển để thay thế cho ASP.NET truyền thống. Với thiết kế linh hoạt và hiệu năng cao, ASP.NET Core hỗ trợ nhiều mô hình triển khai phù hợp với các môi trường khác nhau từ máy chủ nội bộ cho đến các nền tảng đám mây hiện đại.

1.4.1 Môi trường triển khai:

ASP.NET Core có thể triển khai linh hoạt trên nhiều hệ điều hành:

- Windows: Thường sử dụng IIS hoặc self-host thông qua Kestrel kết hợp với IIS làm reverse proxy.
- Linux: Phổ biến với mô hình sử dụng Kestrel làm web server kết hợp với Nginx hoặc Apache reverse proxy.
- macOS: Dùng chủ yếu để phát triển và kiểm thử cục bộ, ít dùng trong sản xuất.

1.4.2 Các hình thức triển khai:

ASP.NET Core hỗ trợ nhiều phương thức triển khai, giúp lập trình viên linh hoạt hơn khi phát hành ứng dụng:

- Framework-dependent deployment (FDD): Máy chủ cần cài sẵn .NET Core Runtime để chạy ứng dụng.
- Self-contained deployment (SCD): Ứng dụng được đóng gói cùng runtime, không phụ thuộc vào hệ thống máy chủ.
- Single-file executable: Toàn bộ ứng dụng được gói gọn trong một tệp thực thi duy nhất (ví dụ: .exe hoặc .dll), tiện lợi cho việc phân phối.

- **Deployment qua CI/CD:** Kết hợp cùng các công cụ tự động hóa như GitHub Actions, Azure DevOps hoặc Jenkins để tự động hóa quy trình build và triển khai.

1.4.3 Các nền tảng đám mây hỗ trợ:

ASP.NET Core tương thích tốt với nhiều nền tảng cloud hiện đại:

- **Microsoft Azure:** Hỗ trợ trực tiếp thông qua Azure App Services, Azure Container Instances và Azure Kubernetes Service.
- **Amazon Web Services (AWS):** Triển khai trên Elastic Beanstalk, EC2, ECS, hoặc Lambda.
- **Google Cloud Platform (GCP):** Có thể chạy trên App Engine, Cloud Run hoặc GKE (Google Kubernetes Engine).
- **Docker & Kubernetes:** ASP.NET Core có thể dễ dàng được container hóa và triển khai trên các nền tảng điều phối container như Docker Swarm hoặc Kubernetes.

1.4.4 Công cụ hỗ trợ triển khai:

Một số công cụ phổ biến được sử dụng để triển khai ứng dụng ASP.NET Core:

- **Visual Studio:** Hỗ trợ publish trực tiếp lên IIS, Azure hoặc thư mục cục bộ.
- **Dotnet CLI:** Cung cấp các lệnh như dotnet publish, dotnet run để build và đóng gói ứng dụng.
- **Docker CLI:** Sử dụng để đóng gói ứng dụng vào container và triển khai trên các môi trường containerized.
- **CI/CD Tools:** Các công cụ như GitHub Actions, GitLab CI/CD hoặc Azure DevOps giúp tự động hóa toàn bộ quy trình triển khai.

CHƯƠNG 2 :XÂY DỰNG ỨNG DỤNG THỰC TẾ

2.1 Phát biểu bài toán ứng dụng:

2.1.1 Giới thiệu dự án:

Trong bối cảnh thương mại điện tử ngày càng phát triển mạnh mẽ, nhu cầu tiếp cận và sở hữu các sản phẩm giải trí, đặc biệt là truyện tranh trực tuyến và bản in, đang ngày càng tăng cao. Đặc biệt, các thể loại truyện như manga (Nhật Bản), manhwa (Hàn Quốc) và manhua (Trung Quốc) đang thu hút một lượng lớn độc giả trẻ tuổi tại Việt Nam cũng như trên toàn thế giới. Tuy nhiên, thị trường vẫn còn thiếu các nền tảng thương mại điện tử chuyên biệt, nơi người dùng có thể dễ dàng tìm kiếm, mua sắm và khám phá những bộ truyện tranh yêu thích, đặc biệt là các ấn phẩm chính hãng, phiên bản giới hạn hoặc dịch vụ đặt trước (pre-order). Chính vì vậy, mình lựa chọn xây dựng một website thương mại điện tử chuyên kinh doanh truyện tranh – phục vụ người yêu truyện một cách tiện lợi, nhanh chóng và hiện đại. Trong những năm gần đây, công nghệ thông tin, đặc biệt là các ứng dụng quản lý web, đang phát triển mạnh mẽ. Mình nhận thấy rõ xu hướng này qua việc các cửa hàng ngày càng tận dụng website bán hàng trực tuyến để tăng doanh thu và tiếp cận khách hàng hiệu quả hơn. Nhận thấy tiềm năng to lớn đó, mình đã quyết định lựa chọn đề tài xây dựng trang website thương mại điện tử kinh doanh truyện tranh anime.

2.1.2 Vai trò của dự án:

Dự án này được xây dựng đóng vai trò là một nền tảng kết nối giữa người bán và người đọc truyện, nơi cung cấp các tính năng chính như:

- Trình bày danh mục truyện theo thể loại và loại truyện (manga, manhwa, manhua...).
- Hệ thống tìm kiếm và lọc nâng cao theo tên, tác giả, thể loại, quốc gia, trạng thái phát hành,...
- Giỏ hàng và thanh toán: Cho phép người dùng thêm truyện vào giỏ hàng, thanh toán bằng chuyển khoản trực tuyến hoặc thanh toán khi nhận hàng.
- Đăng ký/Đăng nhập tài khoản người dùng và theo dõi đơn hàng.

- Thêm truyện tranh yêu thích vào bộ thư viện cá nhân.
- Trang quản trị viên để quản lý sản phẩm (truyện), đơn hàng, người dùng.

2.1.3 Ý nghĩa của dự án:

Việc xây dựng và phát triển dự án này mang lại nhiều ý nghĩa:

- Giúp sinh viên áp dụng kiến thức đã học về HTML, CSS, JavaScript, C#, ASP.NET Core MVC, cơ sở dữ liệu, Entity Framework Core,...
- Rèn luyện kỹ năng lập trình theo mô hình MVC, thiết kế UI/UX, xử lý nghiệp vụ thực tế.
- Giải quyết nhu cầu thiết thực của cộng đồng độc giả yêu thích truyện tranh.
- Hỗ trợ các nhà xuất bản và cửa hàng truyện dễ dàng tiếp cận khách hàng thông qua nền tảng số hóa.
- Góp phần thúc đẩy tiêu dùng văn hóa chính ngạch, hạn chế truyện lậu hoặc bản dịch trái phép.
- Tạo mô hình kinh doanh thử nghiệm có thể mở rộng sau này.
- Tích hợp các dịch vụ như pre-order truyện hot, bán mô hình (figure), phụ kiện liên quan.

2.2 Phân Tích yêu cầu của ứng dụng:

2.2.1 Mô tả chức năng ứng dụng:

Mục đích đề tài xây dựng hệ thống phần mềm hoạt động trên môi trường web để quản lý việc bán hàng được thuận tiện, rất dễ quản lý các sản phẩm giúp cho người quản lý sẽ dễ dàng trong việc quản lý bán hàng cho khách có thể kiểm soát được khách hàng mua hàng và thống kê doanh thu của cửa hàng. Còn khách hàng mua hàng trên web sẽ được thoải mái hơn không tốn thời gian để đến cửa hàng, mà vẫn mua được sản phẩm mà mình thích.

Bộ phận quản lý, thực hiện các nghiệp vụ sau (trước tiên muốn thực hiện các nghiệp vụ cần phải đăng nhập tài khoản và mật khẩu thì mới thực hiện được các

chức năng. Trong đó đã phân quyền các nhân viên để thực hiện các chức năng khác nhau, chỉ có người quản lý chính mới được đầy đủ quyền kiểm soát):

- Xem sản phẩm danh sách sản phẩm mới và danh sách sản phẩm bán chạy.
- Cập nhật giá mới cho sản phẩm.
- Nhập loại danh mục.
- Nhập loại sản phẩm.
- Nhập sản phẩm mới.
- Quản lý người dung
- Xem danh sách người liên hệ.
- Quản lý đơn hàng.

Trang web còn phục vụ cho khách hàng, với những chức năng sau:

- Tìm kiếm sản phẩm theo tên sản phẩm, theo thể loại,(loại truyện).
- Xem sản phẩm.
- Chọn sản phẩm và xem chi tiết sản phẩm.
- Thêm sản phẩm vào giỏ hàng.
- Đặt mua sản phẩm.
- Thanh toán trực tuyến thông minh.
- Xem giỏ hàng đã đặt mua những gì.
- Cập nhật giỏ hàng (Thêm, xóa, cập nhật, xóa tất cả trong giỏ hàng).
- Thêm, xóa sản phẩm vào danh sách yêu thích.
- Đặt hàng (chức năng này cần phải đăng nhập trước).
- Đăng ký tài khoản.
- Đăng nhập tài khoản
- Liên hệ với bên chủ doanh nghiệp thông qua trang liên hệ
- Xem tin nhắn phản hồi từ chủ doanh nghiệp.

Bảng 2.1 chức năng chính của hệ thống đối với quản trị(Admin):

STT	Chức Năng	Mô tả
1	Đăng nhập hệ thống để quản lý	Cho phép quản trị viên đăng nhập vào hệ thống để thực hiện các chức năng quản lý.
2	Nhập sản phẩm mới	Thêm mới thông tin sản phẩm vào hệ thống như tên, mô tả, hình ảnh, loại, giá...
3	Nhập loại sản phẩm	Quản lý danh sách các loại sản phẩm, phục vụ cho việc phân loại và tìm kiếm.
4	Quản lý đơn hàng	Hiển thị, chỉnh sửa, cập nhật và theo dõi trạng thái của các đơn hàng.
5	Thêm giá sản phẩm	Gán giá cho sản phẩm mới hoặc cập nhật giá cho sản phẩm hiện có.
6	Xóa sản phẩm	Xóa sản phẩm không còn kinh doanh khỏi hệ thống.
7	Chỉnh sửa sản phẩm	Cập nhật thông tin sản phẩm như tên, mô tả, hình ảnh, loại sản phẩm, giá...
9	Quản lý khách hàng	Theo dõi thông tin khách hàng, lịch sử mua hàng và hỗ trợ khách hàng.

9	Thanh toán đơn đặt hàng	Thực hiện thao tác thanh toán và cập nhật trạng thái đơn hàng sau khi thanh toán.
10	Xem và phản hồi tin nhắn từ khách hàng	Hệ thống hiển thị tin nhắn khách gửi và cho phép quản trị viên phản hồi.

Bảng 2.2 Chức năng chính của hệ thống đối với khách hàng:

STT	Chức năng	Mô tả
1	Xem sản phẩm	Hiển thị danh sách sản phẩm đang bán trên hệ thống.
2	Đặt hàng	Cho phép người dùng chọn sản phẩm và tiến hành đặt mua.
3	Thêm vào giỏ hàng	Thêm sản phẩm vào giỏ để lưu trữ tạm trước khi đặt hàng.
4	Thêm vào danh sách yêu thích	Lưu sản phẩm yêu thích để dễ dàng tìm lại sau.
5	Gửi thông tin liên hệ	Cho phép người dùng gửi tin nhắn hoặc yêu cầu hỗ trợ.
6	Thanh toán trực tuyến	Hỗ trợ người dùng thanh toán qua cổng thanh toán điện tử.
7	Thanh toán COD	Cho phép thanh toán khi nhận hàng (Cash On Delivery).

8	Quản lý xem giỏ hàng (thêm,xóa, sửa....)	Quản lý nội dung giỏ hàng của người dùng.
9	Đăng ký	Cho phép người dùng tạo tài khoản mới.
10	Đăng nhập	Cho phép người dùng đăng nhập vào hệ thống để sử dụng các chức năng cá nhân.
11	Xem đơn hàng đã đặt	Hiển thị danh sách các đơn hàng mà người dùng đã đặt.
12	Xem thông tin cá nhân	Hiển thị thông tin tài khoản người dùng như tên, email, địa chỉ,...
13	Đổi mật khẩu	Cho phép người dùng thay đổi mật khẩu đăng nhập.
14	Xem nội dung phản hồi từ admin	Hiển thị các phản hồi từ quản trị viên đến người dùng.
15	Xem chi tiết sản phẩm	Hiển thị thông tin chi tiết của sản phẩm như mô tả, giá, đánh giá, hình ảnh,...

2.2.2 Mô tả yêu cầu phi chức năng của ứng dụng:

2.2.2.1 Hiệu suất:

- Tốc độ phản hồi trang web nhanh, thời gian load trang dưới 2 giây trong điều kiện kết nối luôn luôn ổn định.
- Thời gian xử lý giao dịch nhanh nhạy từ click thanh toán đến xác nhận không quá 5 giây.

- Hệ thống phải chịu tải đồng thời ít nhất 500 user truy cập cùng lúc không bị suy giảm hiệu suất.
- Các hình ảnh sản phẩm phải được tối ưu hóa không gây chậm trễ khi hiển thị ảnh sản phẩm.

2.2.2.2 Khả năng sử dụng:

- Giao diện trực quan dễ sử dụng thân thiện với người dùng dễ dàng tìm kiếm sản phẩm để xem và mua.
- Hoạt động tốt trên các trình duyệt, tương thích đa nền tảng (Desktop, Mobile,...).
- Giao diện được responsive tự động điều chỉnh phù hợp với các kích thước màn hình khác nhau.
- Hiển thị thông báo rõ ràng dễ hiểu.

2.2.2.3 Bảo mật:

- Sử dụng ASP.NET Core Identity để quản lý người dùng, đăng nhập, phân quyền.
- Mật khẩu được mã hóa bằng cơ chế Hash tự động.
- Có phân quyền giữa người dùng thường và quản trị viên.

2.2.2.4 Khả năng bảo trì:

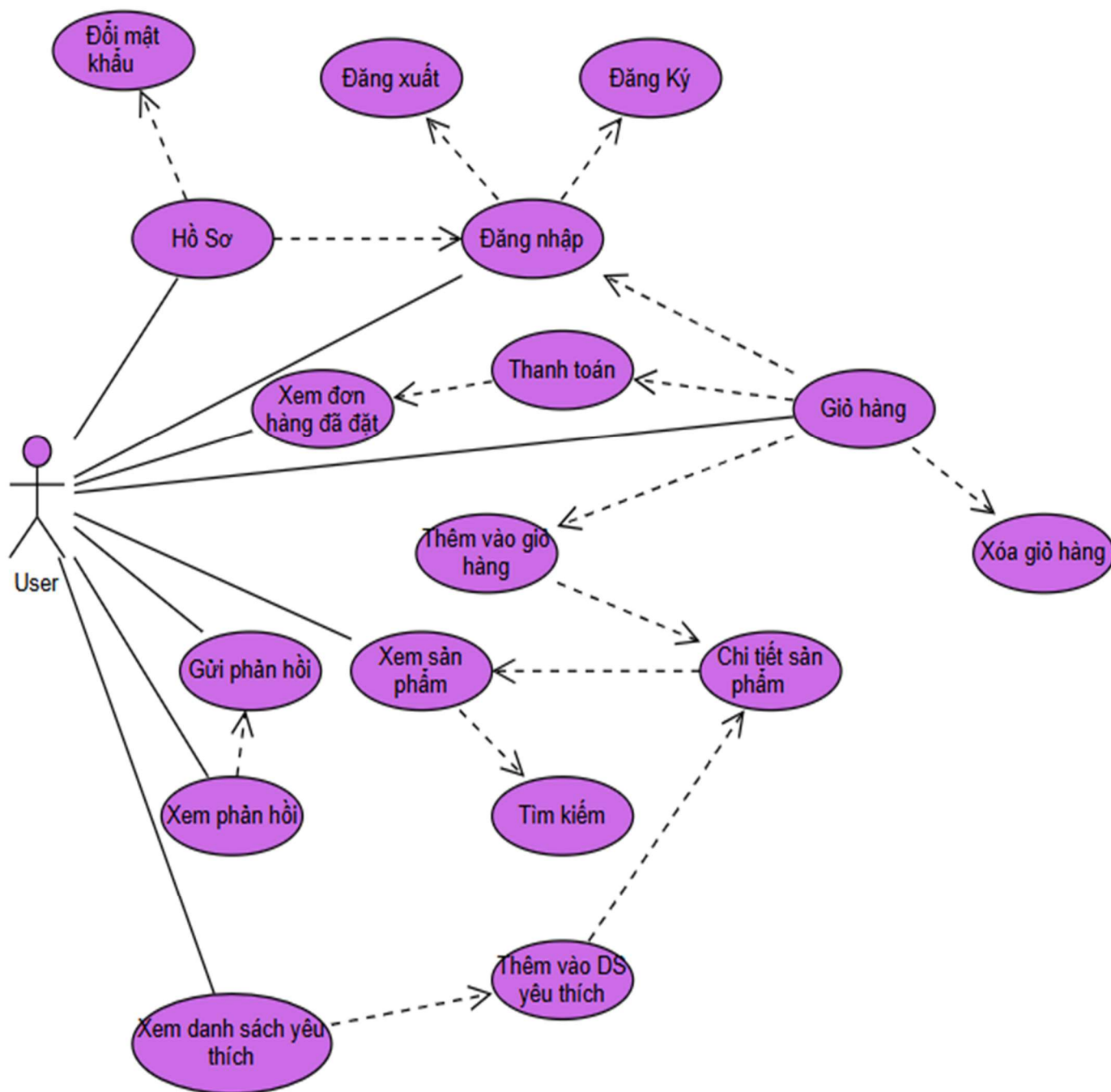
- Code được tổ chức rõ ràng, sử dụng chuẩn ASP.NET MVC.
- Các chức năng được phân tách theo controller, interface, service, dễ bảo trì hoặc chỉnh sửa sau này.

2.2.2.5 Khả năng đáng tin cậy:

- Hệ thống hoạt động trơn tru ít xảy ra lỗi đảm bảo thời gian hoạt động ít nhất 99.5% mỗi tháng.
- Trong trường hợp xảy ra lỗi hệ thống, ứng dụng phải có khả năng tự động phục hồi nhanh chóng và đảm bảo tính toàn vẹn của dữ liệu.
- Dữ liệu hệ thống (thông tin sản phẩm, đơn hàng, người dùng) phải được sao lưu định kỳ và có kế hoạch phục hồi thảm họa.

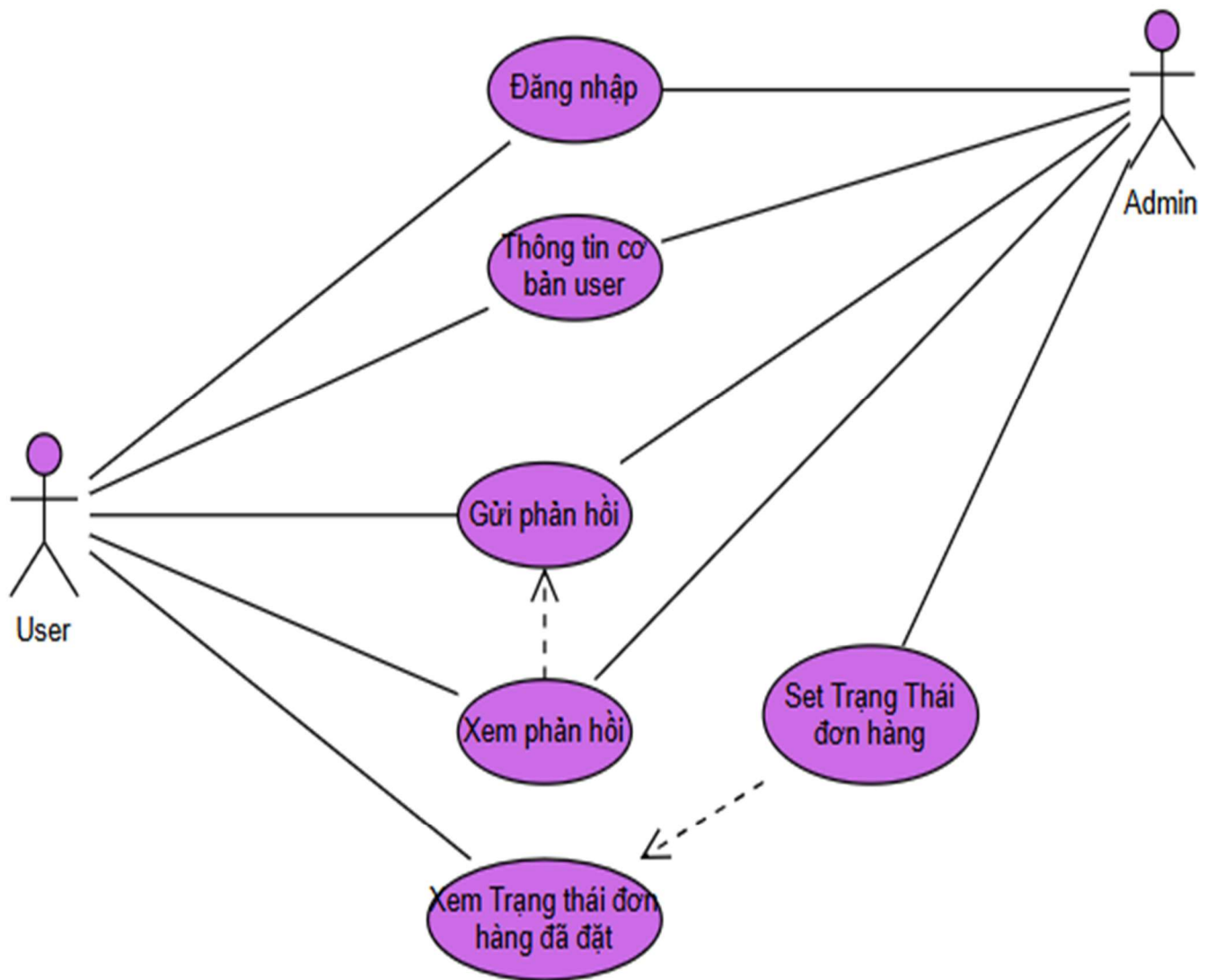
2.2.3 Sơ đồ UseCase:

2.2.3.1 UseCase người dùng (User)



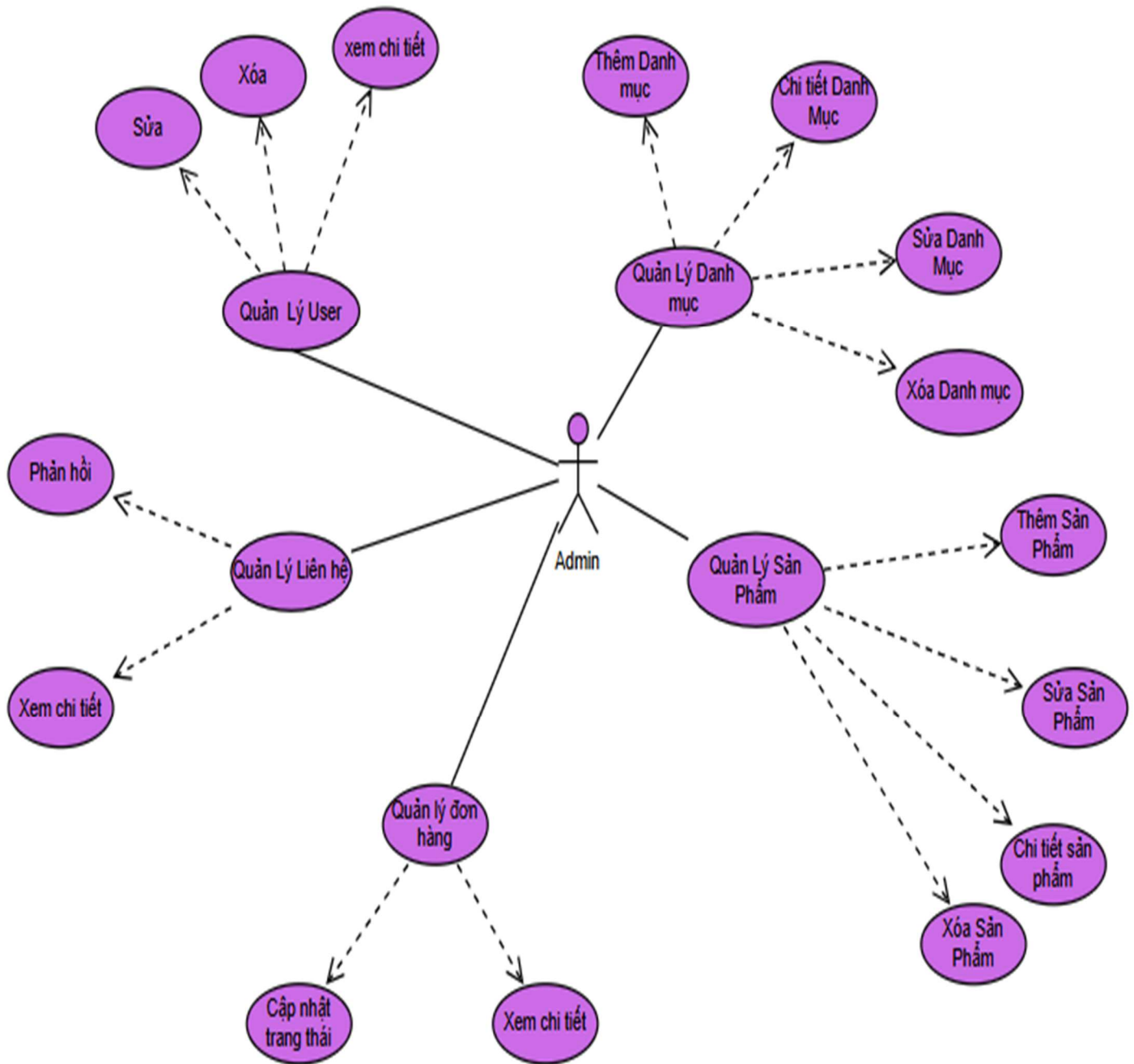
Hình 2.1: Sơ đồ UseCase mô tả chức năng của người dùng (User)

2.2.3.2 UseCase người dùng (User) tương tác với người quản trị (Admin)



Hình 2.2: Sơ đồ UseCase mô tả sự tương tác giữa người dùng(User) với người quản trị(Admin)

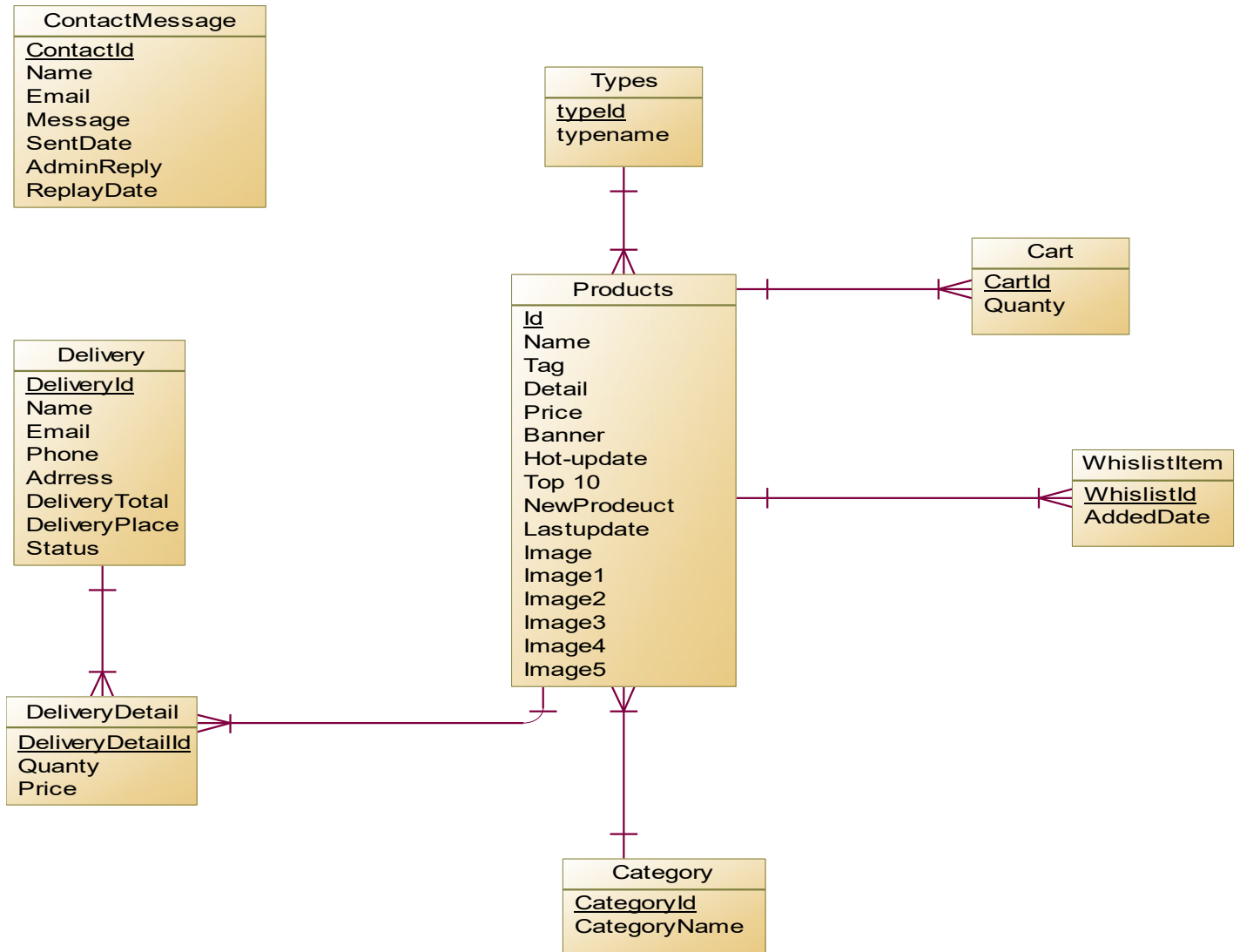
2.2.3.3 UseCase người quản trị viên (Admin)



Hình 2.3: Sơ đồ UseCase mô tả chức năng người quản trị viên(Admin)

2.3 Thiết kế cơ sở dữ liệu:

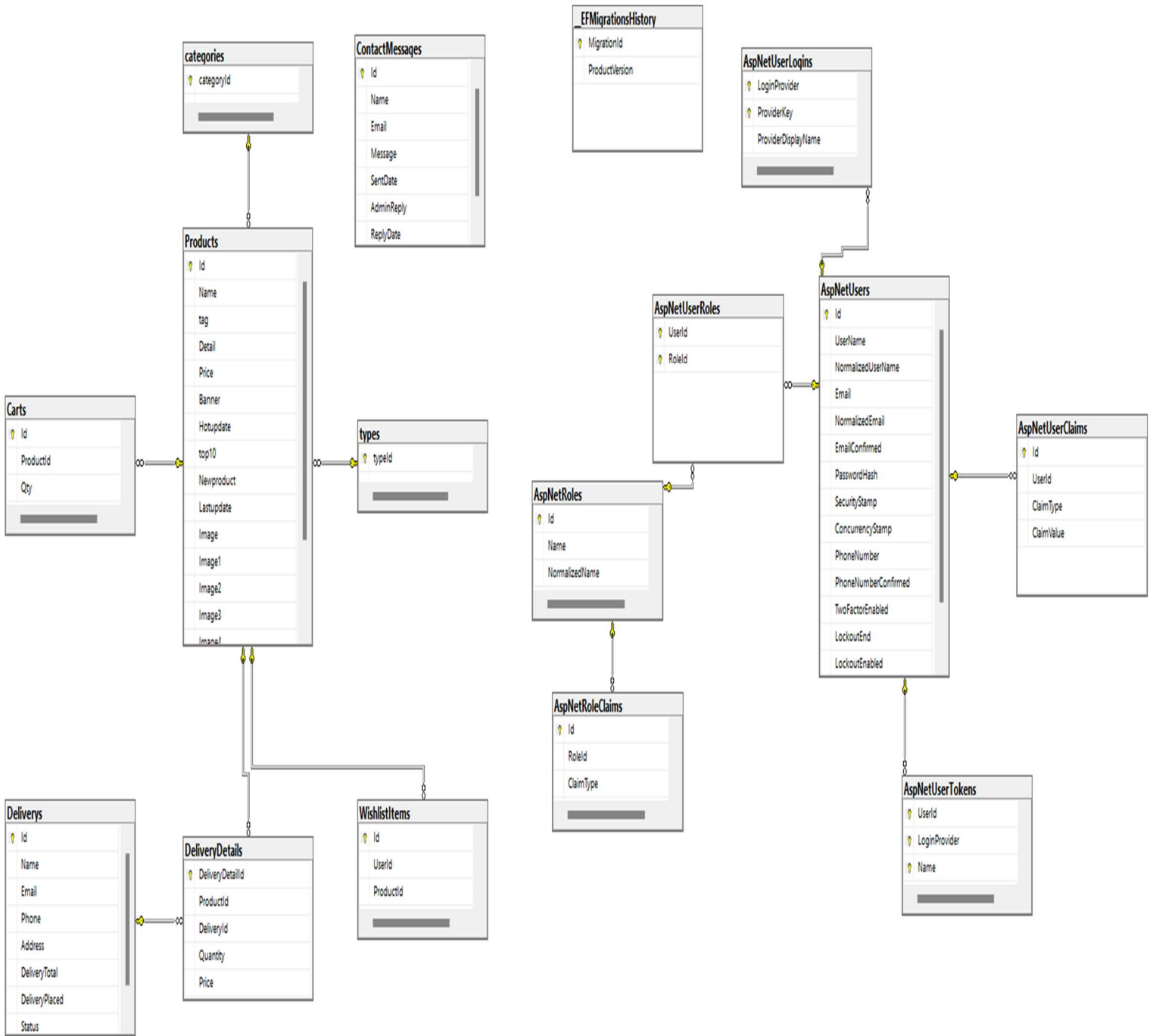
2.3.1 Thiết kế logic ERD:



Hình 2.4: Phần dữ liệu nghiệp vụ (domain data)



2.3.2 Mô hình vật lý



Hình 2.6: Mô hình Cơ sở dữ liệu

2.3.3 Mô tả chi tiết các bảng

a) Bảng dữ liệu nghiệp vụ (domain data)

Bảng 1: Categories

Tên cột	Kiểu Dữ liệu	Mô tả
categoryId	int	Mã định danh duy nhất cho từng danh mục.
categoryname	string	Tên của danh mục (ví dụ: "Shounnen", "Seinen", ...).

Bảng 2: Types

Tên cột	Kiểu Dữ liệu	Mô tả
typeId	int	Mã định danh duy nhất cho từng thể loại.
typename	string	Tên của thể loại (ví dụ: "Manga", "Manhua", ...).

Bảng 3: Products

Tên cột	Kiểu Dữ liệu	Mô tả
Id	Int	Khóa chính, mã định danh duy nhất cho từng sản phẩm.
Name	String	Tên của sản phẩm.
Tag	String	Các thẻ (tags) liên quan đến sản phẩm, phục vụ mục đích phân loại hoặc tìm kiếm.
Detail	String	Mô tả chi tiết về sản phẩm.
Price	Decimal	Giá của sản phẩm.
Banner	Bool	Chỉ định sản phẩm có được hiển thị trên banner quảng cáo hay không.
Hotupdate	Bool	Chỉ định sản phẩm là mới nổi.
Top 10	Bool	Chỉ định sản phẩm có nằm trong danh sách "Top 10" sản phẩm nổi bật hay không.
Newproduct	Bool	Chỉ định sản phẩm là sản phẩm mới ra mắt.

Lastupdate	Bool	Chỉ định sản phẩm có được cập nhật gần đây không.
Image	String	Hiển thị ảnh chính của sản phẩm
Image1	String	Hiển thị ảnh phụ 1 của sản phẩm
Image2	String	Hiển thị ảnh phụ 2 của sản phẩm
Image3	String	Hiển thị ảnh phụ 3 của sản phẩm
Image4	String	Hiển thị ảnh phụ 4 của sản phẩm
Image5	String	Hiển thị ảnh phụ 5 của sản phẩm

Bảng 4: Carts

Tên cột	Kiểu Dữ liệu	Mô tả
CartId	Int	Mã định danh của giỏ hàng
Qty	Int	Số lượng sản phẩm trong giỏ hàng.
Price	Decimal	Giá của sản phẩm tại thời điểm thêm vào giỏ hàng (hoặc giá đơn vị của sản phẩm).

Bảng 5: Deliverys

Tên cột	Kiểu Dữ liệu	Mô tả
DeliveryId	Int	Khóa chính, mã định danh duy nhất cho thông tin giao hàng hoặc đơn hàng.
Name	String	Tên người nhận hàng.
Email	String	Địa chỉ email liên hệ của người nhận.
Phone	String	Số điện thoại liên hệ của người nhận.
Address	String	Địa chỉ giao hàng.
DeliveryTotal	Decimal	Tổng giá trị của đơn hàng.
Deliverypalced	Datetime	Thời điểm đặt hàng.
Status	String	Trạng thái của đơn hàng hoặc quá trình giao hàng (ví dụ: "Đang xử lý", "Đã giao", "Đã hủy").

Bảng 6: DeliverysDetails

Tên cột	Kiểu Dữ liệu	Mô tả
Quantity	Int	Số lượng đơn hàng
Price	Int	Giá trị đơn hàng

Bảng 7: WhislistItems

Tên cột	Kiểu Dữ liệu	Mô tả
Id	Int	Khóa chính, mã định danh duy nhất
Addeddate	Datetime	Thời gian thêm vào danh sách yêu thích

Bảng 8: Contacts Message

Tên cột	Kiểu Dữ liệu	Mô tả
Id	Int	Khóa chính, mã định danh duy nhất cho từng tin nhắn/yêu cầu liên hệ.
Name	String	Tên của người gửi tin nhắn/yêu cầu.
Email	String	Địa chỉ email của người gửi.
Message	Sring	Nội dung tin nhắn
Sentdate	Datetime	Thời gian tin nhắn được gửi đi.
AdminReply	String	Nội dung phản hồi từ quản trị viên (admin) đối với tin nhắn này.
ReplyDate	Datetime	Thời gian quản trị viên phản hồi tin nhắn.

b) Bảng dữ liệu Identity

Không do mình tạo ra thủ công như bảng nghiệp vụ, mình sử dụng thư viện ASP.NET Core Identity để xử lý việc đăng nhập, đăng ký, và phân quyền người dùng. Khi thực hiện migration, Identity tự động tạo các bảng như sau:

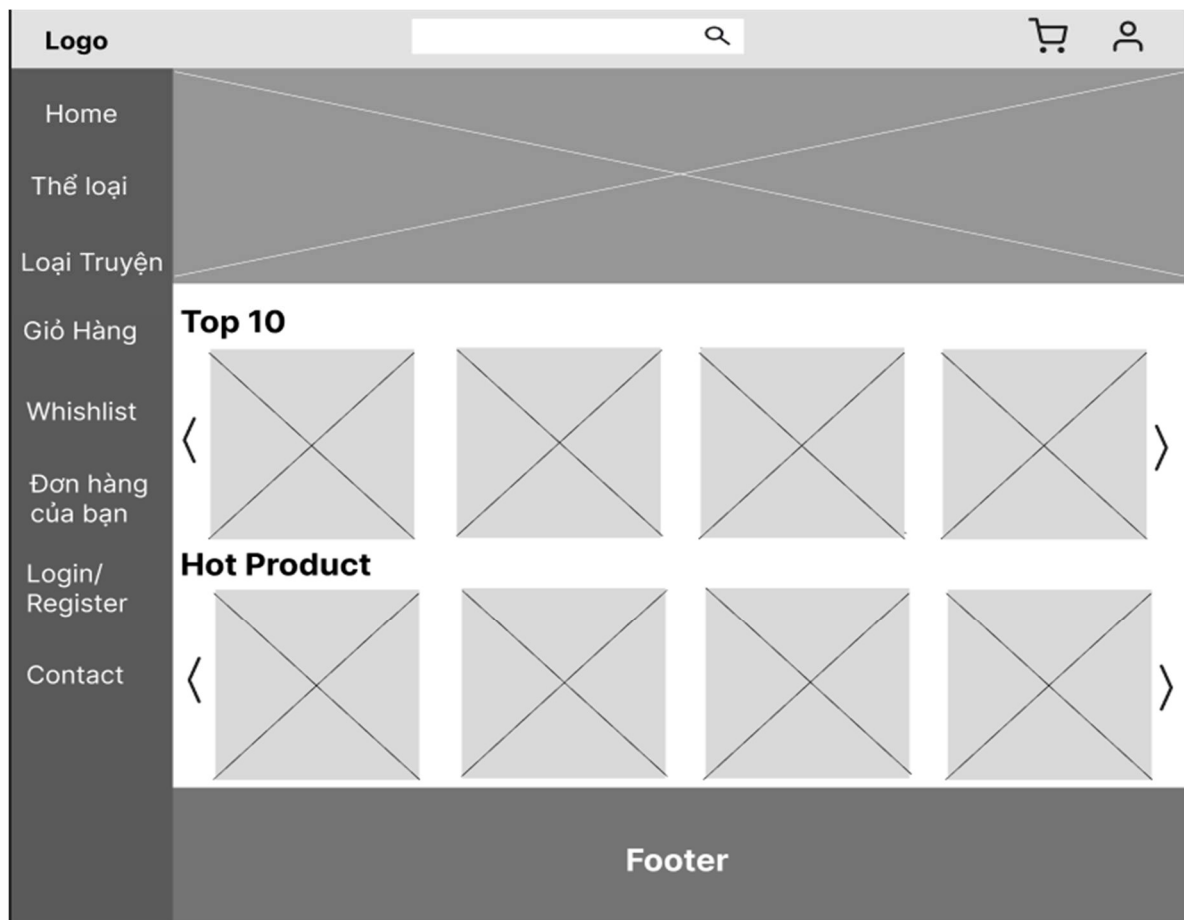
- AspNetUser: chứa thông tin người dùng (username, email, mật khẩu mã hóa,...). Trong hệ thống này, bảng này được mở rộng thêm các trường như FullName, Balance để phục vụ mục đích nghiệp vụ.

- AspNetRoles: lưu các vai trò người dùng như Admin, Khách hàng,...
- AspNetUserRoles: ánh xạ nhiều-nhiều giữa người dùng và vai trò.
- Các bảng hỗ trợ khác như AspNetUserClaims, AspNetUserLogins, AspNetUserTokens phục vụ cho các chức năng xác thực và phân quyền nâng cao.

2.4 Thiết kế giao diện:

2.4.1 Trang chủ:

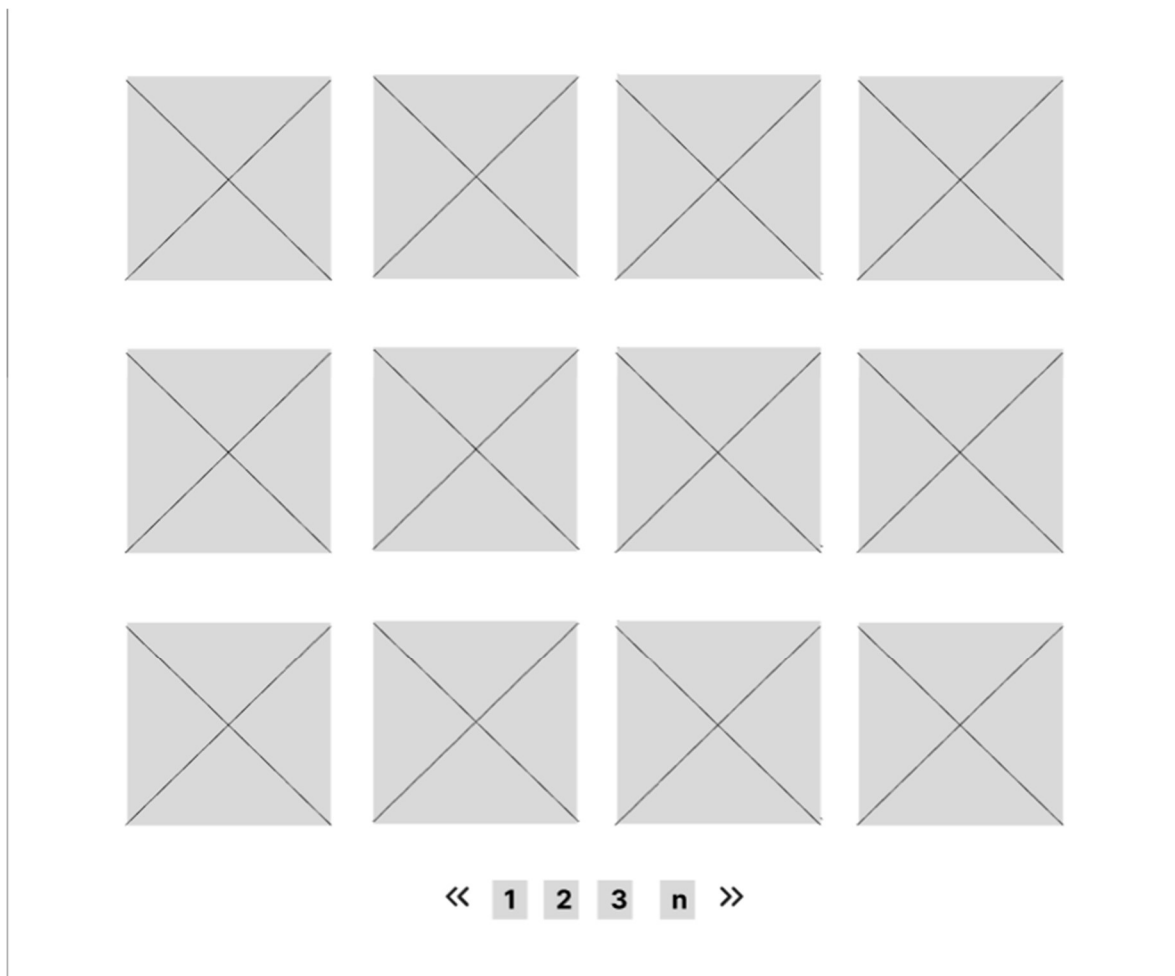
Hiển thị banner, sản phẩm nổi bật, tìm kiếm nhanh, menu danh mục.



Hình 2.7: Bản wireframe giao diện trang chủ

2.4.2 Trang danh mục (Thẻ loại / Loại truyện/ Whishlist)

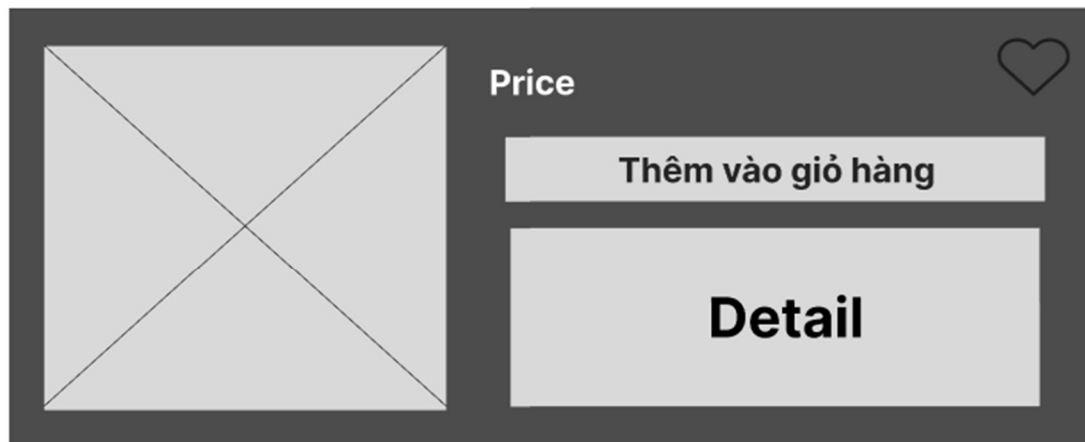
Hiển thị danh sách theo danh mục, các truyện theo thẻ loại, danh sách yêu thích.



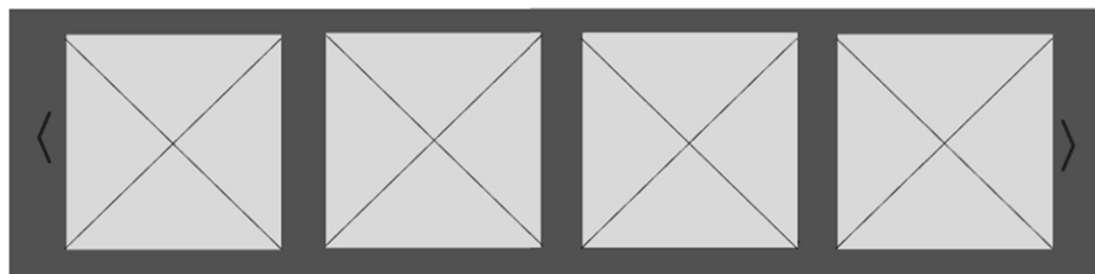
Hình 2.8: Bản wireframe giao diện trang danh mục/whishlist

2.4.3 Trang chi tiết sản phẩm

Hiển thị thông tin chi tiết một truyện: tên, mô tả, hình ảnh, giá, thêm vào giỏ hàng.



Preview



Hình 2.9: Bản wireframe giao diện trang chi tiết sản phẩm

2.4.4 Giỏ hàng

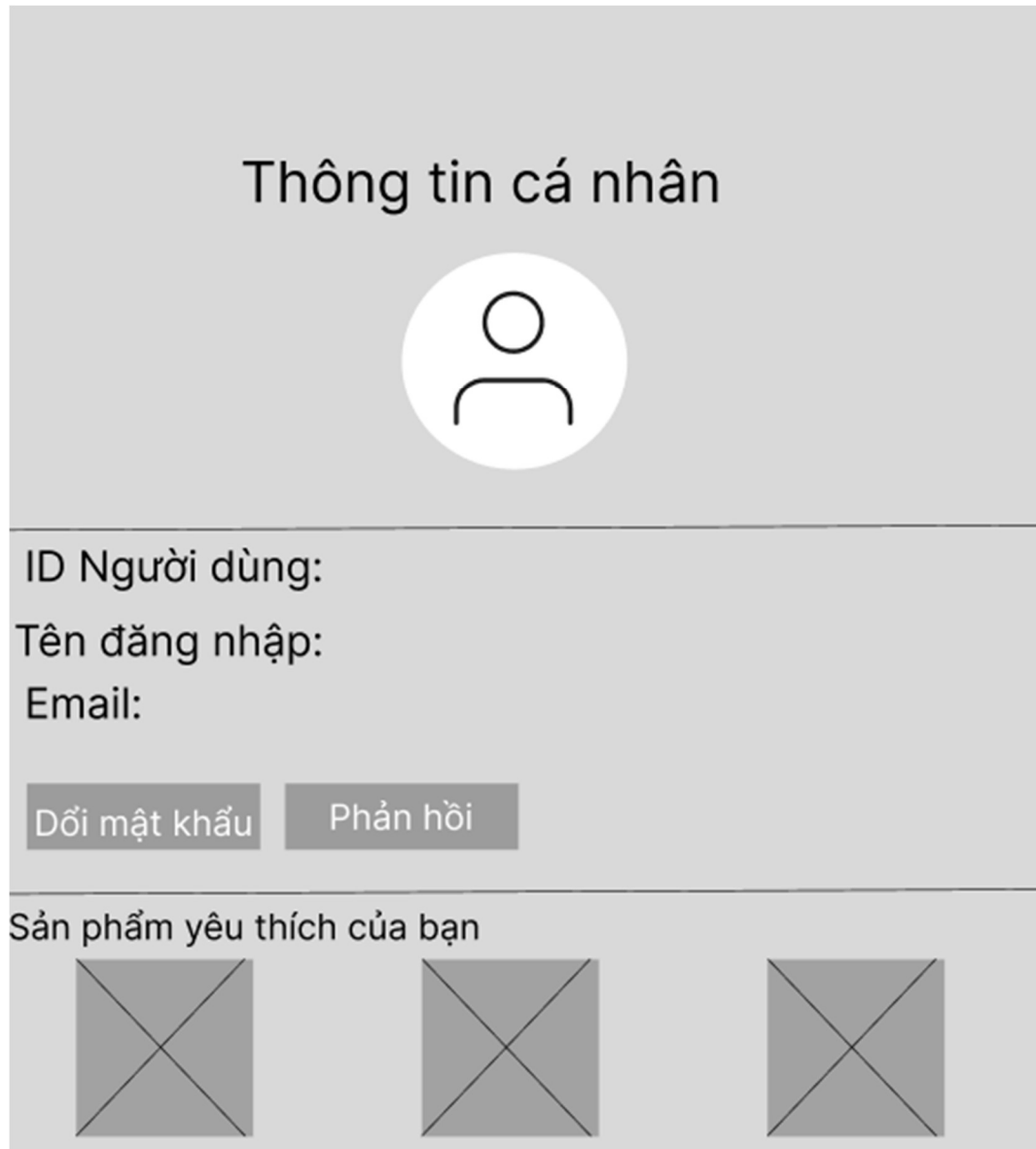
Danh sách sản phẩm đã thêm, cập nhật số lượng, xóa sản phẩm, tổng tiền.



Hình 2.10: Bản wireframe trang giỏ hàng

2.4.5 Profile

Hiển thị thông tin cá nhân của người dùng.



Thông tin cá nhân

ID Người dùng:
Tên đăng nhập:
Email:

Đổi mật khẩu Phản hồi

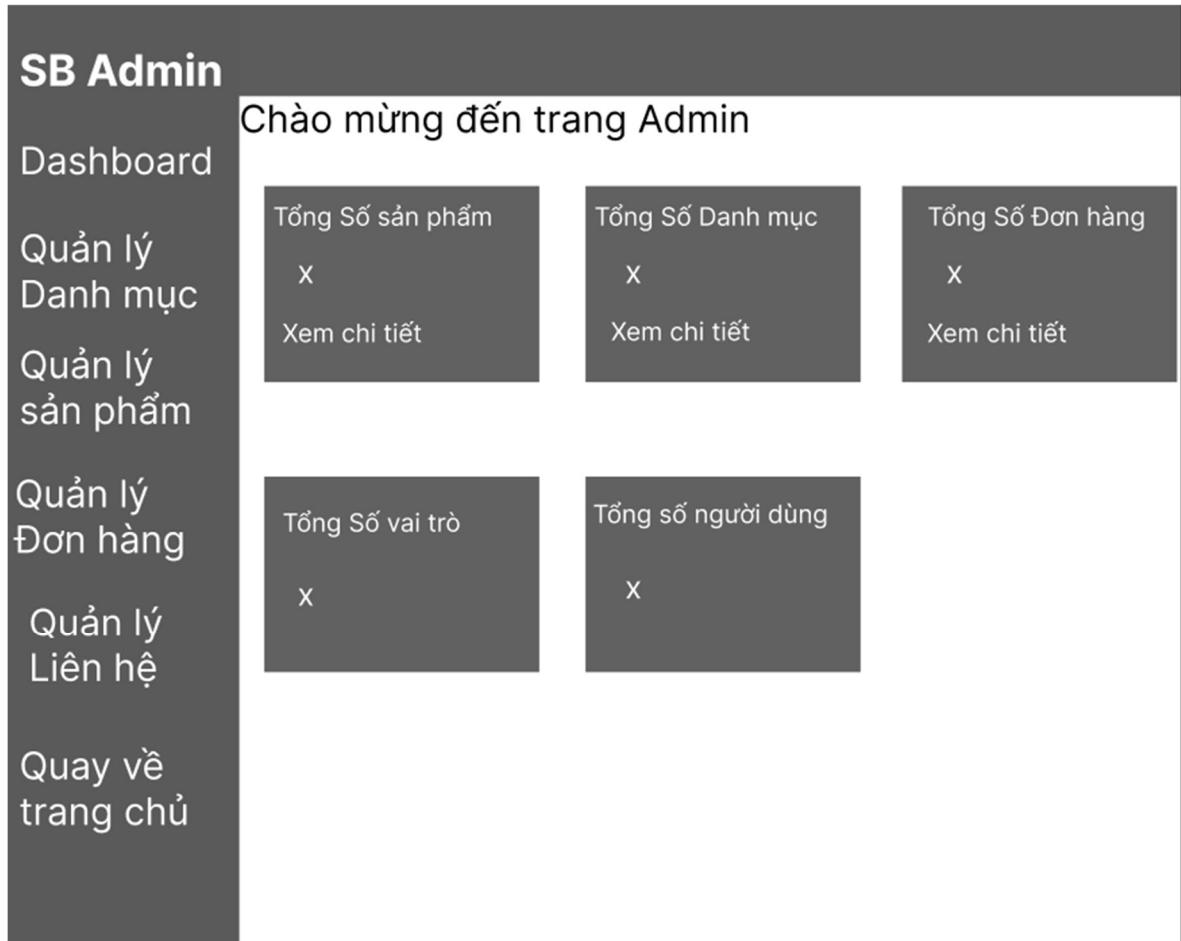
Sản phẩm yêu thích của bạn

Hình 2.11: Bản wiframe trang profile

2.4.6 Trang Admin

- Dashboard (Trang tổng quan): Thống kê nhanh: số đơn hàng, số người dùng, doanh thu...
- Quản lý sản phẩm: Xem, thêm, sửa, xóa truyện tranh
- Quản lý danh mục: Tạo, sửa, xóa danh mục truyện

- Quản lý đơn hàng: Danh sách đơn hàng, trạng thái, xử lý đơn
- Quản lý người dùng: Danh sách tài khoản, phân quyền
- Quản lý liên hệ (Contact messages): Xem và phản hồi các góp ý từ người dùng.



Hình 2.12: Bản wireframe Trang Admin

2.5 Thiết kế các thành phần MVC:

2.5.1 Model :

Tầng Model trong kiến trúc MVC chịu trách nhiệm quản lý dữ liệu, định nghĩa cấu trúc của dữ liệu và các quy tắc nghiệp vụ liên quan. Nó tương tác trực tiếp với cơ sở dữ liệu để lấy, lưu trữ, cập nhật và xóa dữ liệu. Trong ứng dụng, tầng

Model được triển khai dưới dạng các lớp C# (POCOs - Plain Old CLR Objects) kết hợp với Entity Framework Core để ánh xạ với các bảng trong cơ sở dữ liệu.

2.5.1.1 Class Product

```
public class Product
{
    50 references
    public int Id { get; set; }
    44 references
    public string? Name { get; set; }
    18 references
    public string? tag { get; set; }
    23 references
    public string? Detail { get; set; }
    27 references
    public decimal Price { get; set; }
    20 references
    public bool Banner { get; set; }
    20 references
    public bool Hotupdate { get; set; }
    20 references
    public bool topl0 { get; set; }
    20 references
    public bool Newproduct { get; set; }
    20 references
    public bool Lastupdate { get; set; }
    42 references
    public string? Image { get; set; }
    22 references
    public string? Image1 { get; set; }
    22 references
    public string? Image2 { get; set; }
    22 references
    public string? Image3 { get; set; }
    22 references
    public string? Image4 { get; set; }
    22 references
    public string? Image5 { get; set; }
    21 references
    public int categoryId { get; set; }
    8 references
    public category? category { get; set; }
    21 references
    public int typeId { get; set; }
    8 references
    public type? type { get; set; }
}
```

Mô tả chi tiết xem ở Bảng 3

2.5.1.2 Class Category

```
11 references
public class category
{
    14 references
    public int categoryId { get; set; }
    15 references
    public string? categoryName { get; set; }
    1 reference
    public List<Product> Products { get; set; }
}
```

Mô tả chi tiết xem ở Bảng 1

2.5.1.3 Class Type

```
public class type
{
    13 references
    public int typeId { get; set; }
    14 references
    public string? typename { get; set; }

    1 reference
    public List<Product> Products { get; set; }
}
```

Mô tả chi tiết xem ở Bảng 2

2.5.1.4 Class Cart

```
public class Cart
{
    0 references
    public int Id { get; set; }

    16 references
    public Product? Product { get; set; }
    9 references
    public int Qty { get; set; }
    8 references
    public string? CartId { get; set; }
}
```

Mô tả chi tiết xem ở Bảng 4

2.5.1.5 Class Contact

```
public class Contact
{
    9 references
    public int Id { get; set; }

    15 references
    public string Name { get; set; }

    10 references
    public string Email { get; set; }

    14 references
    public string Message { get; set; }

    11 references
    public DateTime SentDate { get; set; } = DateTime.Now;
    13 references
    public string? AdminReply { get; set; }
    8 references
    public DateTime? ReplyDate { get; set; }
}
```

Mô tả chi tiết xem ở Bảng 8

2.5.1.6 Class WishlistItem

```
public class WishlistItem
{
    0 references
    public int Id { get; set; }

    7 references
    public string UserId { get; set; }

    5 references
    public int ProductId { get; set; }
    7 references
    public virtual Product Product { get; set; }

    1 reference
    public DateTime AddedDate { get; set; } = DateTime.Now;
}
```

Mô tả chi tiết xem ở Bảng 7

2.5.1.7 Class Delivery

```
public class Delivery
{
    - references
    public int Id { get; set; }
    - references
    public string? Name { get; set; }
    8 references
    public string? Email { get; set; }
    8 references
    public string? Phone { get; set; }
    7 references
    public string? Address { get; set; }
    8 references
    public decimal DeliveryTotal { get; set; }
    11 references
    public DateTime DeliveryPlaced { get; set; }
    15 references
    public string Status { get; set; } = "Pending";
    11 references
    public List<DeliveryDetail>? DeliveryDetails { get; set; }
}
```

Mô tả chi tiết xem ở Bảng 5

2.5.1.8 Class DeliveryDetail

```
public class DeliveryDetail
{
    public int DeliveryDetailId { get; set; }
    1 reference
    public int ProductId { get; set; }
    7 references
    public Product? Product { get; set; }
    0 references
    public int DeliveryId { get; set; }
    0 references
    public Delivery? Order { get; set; }
    4 references
    public int Quantity { get; set; }
    4 references
    public decimal Price { get; set; }
}
```

Mô tả chi tiết xem ở Bảng 6

2.5.2 View

View là phần giao diện hiển thị dữ liệu từ model và cho phép người dùng tương tác với hệ thống. Trong dự án này các View được mình xây dựng bằng công nghệ HTML/CSS kết hợp với Razor và các framework giao diện như Bootstrap để đảm bảo tính thẩm mỹ, phản hồi nhanh và thân thiện với người dùng.

2.5.2.1 Home

Được hiển thị theo wireframe ở **Hình 2.7**, trang đầu hiển thị các mục banner, các sản phẩm bán chạy,... sử dụng vòng lặp foreach để hiển thị các dữ liệu ra view

```
@foreach (var item in Model)
{
    <div class="slide1">
        <div class="one-piece-banner" style="background-image: url(@item.Image);">
            <div class="overlay"></div>
            <div class="content">
                <h1>@item.Name</h1>
                <div class="tags">@item.tag</div>
                <p>
                    @item.Detail
                </p>
                <a asp-controller="Home" asp-action="Detail" asp-route-id="@item.Id"><button>More info</button></a>
            </div>
        </div>
    </div>
}
```

2.5.2.2 Chi tiết sản phẩm

Được hiển thị theo wireframe ở **Hình 2.9**, trang đầu hiển thị các chi tiết của sản phẩm như giá,... khác với Home trang này chỉ hiển thị duy nhất 1 sp ko cần vòng lặp.

```
<div class="info-sp" style="background-color: #f2f2f2; border-radius: 20px; margin-top: 50px; position: relative;">
    <div class="sp">
        
    </div>
    <div class="info">
        <div style="display: flex; justify-content: space-between;">
            <h2 style="font-weight: bold;">@Model.Product.Price.ToString("N0")đ</h2>
            <div>
                <i class="fa-solid fa-heart wishlist-inline-icon" data-product-id="@Model.Product.Id" style="cursor: pointer; font-size: 1.8rem; color: @(Model.IsInWishlist ? "blueviolet" : "#ccc"); position: absolute; top: 20px; right: 20px; z-index: 10;">
                </i>
            </div>
        </div>
        <a asp-controller="Cart" asp-action="AddCart" asp-route-pId="@Model.Product.Id">
            <button style="padding: 5px 10px; border-radius: 10px; width: 100%;">
                Thêm vào giỏ
            </button>
        </a>
        <div><p>@Model.Product.tag </p></div>
        <p>
            @Model.Product.Detail
        </p>
    </div>
</div>
```

2.5.2.3 Giỏ hàng

Được hiển thị theo wireframe ở **Hình 2.10**, nếu giỏ hàng trống sẽ hiển thị thông báo, nếu model != 0 thì sẽ hiển thị sản phẩm được thêm vào giỏ hàng thông qua logic được xử lý trong controller có thể tăng số lượng hoặc xóa giỏ hàng.

```
@if (Model.Count() != 0)
{
    <div class="mapcart">
    <div class="cart">
    <h3>Giỏ Hàng của bạn </h3>
    <span>Bạn có @Model.Count() sản phẩm</span>
    @foreach (var item in Model)
    {
        <div class="item">
        
        <span class="name">@item.Product.Name</span>
        <span class="price">@item.Product.Price.ToString("N0")đ</span>
        <div class="plus">
        <form asp-controller="Cart" asp-action="IncreaseQty" method="post" style="display:inline;">
        <input type="hidden" name="productId" value="@item.Product.Id" />
        <button id="plus" type="submit" style="background:none; border:none; padding:0; cursor:pointer;">
        <i class="fa-solid fa-angle-up" style="color: #000000;"></i>
        </button>
        </form>
        <input type="text" value="@item.Qty" readonly style="width: 30px; text-align: center; border: 0; background: none; color: #000000;" />
        <form asp-controller="Cart" asp-action="DecreaseQty" method="post" style="display:inline;">
        <input type="hidden" name="productId" value="@item.Product.Id" />
        <button id="minus" type="submit" style="background:none; border:none; padding:0; cursor:pointer;">
        <i class="fa-solid fa-angle-down" style="color: #000000;"></i>
        </button>
        </form>
        </div>
        <div><a asp-controller="Cart" asp-action="RemoveCart" asp-route-pld="@item.Product.Id"><i id="remove" class="fa-solid fa-delete-left"></i></a></div>
    </div>
    }
    @{
        decimal grandTotal = 0;
        if (Model != null)
        {
            foreach (var item in Model)
            {
                grandTotal += (item.Qty * item.Product.Price);
            }
        }
    }
    <div class="sum">
    <div class="left">
    <p>Tổng Sản Phẩm</p>
    <p>Thanh Toán</p>
    </div>
    <div class="right">
    <p>@grandTotal.ToString("N0")đ</p>
    <p>@grandTotal.ToString("N0")đ</p>
    </div>
    <a asp-controller="Delivery" asp-action="Checkout">
    <button>Thanh toán</button>
    </a>
    </div>
}
else
{
    <div class="empty">
    <h3>Giỏ Hàng của bạn đang trống !!</h3>
    
    <div>
    <a asp-controller="Home" asp-action="Index"><button>Quay lại Trang chủ</button></a>
    </div>
    </div>
}
```

2.5.2.4 Danh mục thể loại

Được hiển thị theo wireframe ở **Hình 2.8**, tương tự như home view này cũng sử dụng vòng lặp để hiển thị số lượng lớn các sản phẩm lấy trong database, view này sẽ đc giới hạn số lượng sp hiển thị và sẽ đc chuyển sang trang 2 nếu trang 1 đầy thông qua js.

```

<div class="wrap">
  <div class="slider-contain">
    <div class="slides">
      @foreach (var item in Model)
      {
        <a style="text-decoration: none;" asp-controller="Home" asp-action="Detail" asp-route-id="@item.Id"><div class="cards"><p>@item.Name</p></div></a>
      }
    </div>
  </div>
</div>
<div class="cens">
  <!-- Mút số trang sẽ tự tạo bằng JS -->
  <button class="prev"><i class="fa-solid fa-angle-left"></i></button>
  <span class="page-numbers" style="color: #fff;"></span>
  <button class="next"><i class="fa-solid fa-angle-right"></i></button>
</div>

```

Cấu trúc hiển thị các view còn lại tương tự như các view trên

2.5.3 Controller

Controller là thành phần trung gian giữa Model và View. Nó tiếp nhận yêu cầu (request) từ người dùng, xử lý logic phù hợp bằng cách sử dụng dữ liệu từ Model, sau đó trả về kết quả thông qua View. Controller giúp quản lý luồng điều khiển của ứng dụng và định nghĩa các hành động mà người dùng có thể thực hiện.

Trong hệ thống, mỗi Controller thường tương ứng với một nhóm chức năng chính của ứng dụng, ví dụ như quản lý bài viết, người dùng, sản phẩm, đơn hàng.

2.5.3.1 Hiển thị danh sách

Lấy danh sách được chỉ định xuất ra rồi truyền sang view để hiển thị

```

public IActionResult Index()
{
    ViewBag.HotUpdate = _productRepository.GetHotupdate();
    ViewBag.Top10 = _productRepository.Gettop10();
    ViewBag.NewProduct = _productRepository.GetNewproduct();
    ViewBag.LastUpdate = _productRepository.GetLastupdate();

    return View(_productRepository.GetBanner());
}

```

```

2 references
public IEnumerable<Product> GetBanner()
{
    return db.Products.Where(p => p.Banner);
}

2 references
public IEnumerable<Product> GetHotupdate()
{
    return db.Products.Where(p => p.Hotupdate);
}

2 references
public IEnumerable<Product> Gettop10()
{
    return db.Products.Where(p => p.top10);
}

2 references
public IEnumerable<Product> GetNewproduct()
{
    return db.Products.Where(p => p.Newproduct);
}

2 references
public IEnumerable<Product> GetLastupdate()
{
    return db.Products.Where(p => p.Lastupdate);
}

```

2.5.3.2 *Hiển thị chi tiết sản phẩm*

Tại đây khi click bất kì một đối tượng sp nào được hiển thị ra view thì phương thức này sẽ gọi đến id của đối tượng đó sau đó sẽ hiển thị ra view chi tiết sản phẩm theo id vừa click

```

public async Task<IActionResult> Detail(int id)
{
    var product = _productRepository.GetProductDetail(id);
    if (product == null)
    {
        return NotFound();
    }

    //var viewModel = new ProductDetailViewModel
    //...;

    // if (User.Identity.IsAuthenticated)
    //...

    return View(product);
}

```

```

2 references
public Product? GetProductDetail(int id)
{
    return db.Products.FirstOrDefault(p => p.Id == id);
}

```

2.5.3.3 Thêm/xóa vào danh sách yêu thích

Tại đây tương tự như detail sẽ truy vấn đến id của đối tượng của sản phẩm và được insert hoặc delete khỏi bảng wishlist, để sử dụng phương thức này cần phải đăng nhập [Authorize].

```

public async Task<ActionResult> ToggleWishlist(int productId)
{
    if (!User.Identity.IsAuthenticated)
    {
        return Json(new { success = false, message = "Bạn cần đăng nhập để thêm sản phẩm vào danh sách yêu thích.", redirectToLogin = true });
    }

    var userId = _userManager.GetUserId(User);
    if (userId == null)
    {
        return Json(new { success = false, message = "Không thể tìm thấy thông tin người dùng." });
    }

    bool isAdded = await _productRepository.ToggleWishlistAsync(productId, userId);

    return Json(new { success = true, isAdded = isAdded, message = isAdded ? "Đã thêm vào danh sách yêu thích!" : "Đã xóa khỏi danh sách yêu thích." });
}

0 references
public async Task<ActionResult> Wishlist()
{
    var userId = _userManager.GetUserId(User);

    if (string.IsNullOrEmpty(userId))
    {
        TempData["Message"] = "Vui lòng đăng nhập để xem danh sách yêu thích của bạn.";
        return Redirect("/Identity/Account/Login");
    }

    var productsInWishlist = await _productRepository.GetProductsInWishlistAsync(userId);

    return View(productsInWishlist);
}

2 references
public async Task<bool> ToggleWishlistAsync(int productId, string userId)
{
    if (string.IsNullOrEmpty(userId))
    {
        return false;
    }

    var existingItem = await db.WishlistItems
        .FirstOrDefaultAsync(wi => wi.ProductId == productId && wi.UserId == userId);

    if (existingItem != null)
    {
        db.WishlistItems.Remove(existingItem);
        await db.SaveChangesAsync();
        return false;
    }
    else
    {
        var wishlistItem = new WishlistItem
        {
            ProductId = productId,
            UserId = userId,
            AddedDate = DateTime.Now
        };
        db.WishlistItems.Add(wishlistItem);
        await db.SaveChangesAsync();
        return true;
    }
}

```

2.5.3.4 Tìm kiếm

Phương thức này đơn giản chỉ là truy vấn thông thường theo tên hoặc theo id,...


```

0 references
public async Task<IActionResult> Search(string searchTerm)
{
    ViewBag.CurrentSearchTerm = searchTerm;

    var searchResults = await _productRepository.SearchProductsAsync(searchTerm);

    if (searchResults == null || !searchResults.Any())
    {
        ViewBag.NoResultsMessage = $"Không tìm thấy sản phẩm nào khớp với từ khóa '{searchTerm}'.";
    }

    return View(searchResults);
}

```

```

2 references
public async Task<List<Product>> SearchProductsAsync(string searchTerm)
{
    if (string.IsNullOrEmpty(searchTerm))
    {
        return new List<Product>();
    }

    return await db.Products
        .Where(p => p.Name.Contains(searchTerm) || p.Detail.Contains(searchTerm))
        .ToListAsync();
}

```

2.5.3.5 Thêm/Xóa giỏ hàng

Đối với phương thức này sẽ sử dụng thêm session để định nghĩa dữ liệu tạm thời cho cart number, có thể hiểu như sau khi sản phẩm được thêm vào giỏ hàng thì cartnumber sẽ ++ và tương tự ngược lại khi giỏ hàng đc thanh toán và trống

```

0 references
public RedirectToActionResult AddCart(int pId)
{
    var product = productRepository.GetAllProducts().FirstOrDefault(p => p.Id ==
pId);
    if (product != null)
    {
        CartRepository.AddCart(product);
        int cartCount = CartRepository.GetAllCartItems().Count();
        HttpContext.Session.SetInt32("CartCount", cartCount);
    }
    return RedirectToAction("Index");
}

0 references
public RedirectToActionResult RemoveCart(int pId)
{
    var product = productRepository.GetAllProducts().FirstOrDefault(p => p.Id ==
pId);
    if (product != null)
    {
        CartRepository.RemoveCart(product);
        int cartCount = CartRepository.GetAllCartItems().Count();
        HttpContext.Session.SetInt32("CartCount", cartCount);
    }
    return RedirectToAction("Index");
}

```

2.5.3.6 Thêm/ sửa/ xóa (Admin Role)

Đối với phương thức này người duy nhất có thể sử dụng được chỉ có thể là admin được xác thực bằng câu lệnh `[Authorize(Roles = "Admin")]` ngăn không cho bất kỳ role nào khác truy cập được.

```
[Area("Admin")]
[Authorize(Roles = "Admin")] // Chỉ cho phép Admin truy cập

1 reference
0 references
public IActionResult Create()
{
    ViewData["categoryId"] = new SelectList(_context.categories, "categoryId", "categoryname");
    ViewData["typeId"] = new SelectList(_context.types, "typeId", "typename");
    return View();
}

// POST: Admin/AdminProducts/Create (Xử lý lưu sản phẩm mới)
[HttpPost]
[ValidateAntiForgeryToken]
// Sửa Bind để bao gồm tất cả các Image properties
0 references
public async Task<IActionResult> Create(
    [Bind("Id,Name,tag,Detail,Price,Banner,Hotupdate,top10,Newproduct,Lastupdate,categoryId,typeId")] Product product,
    IFormFile imageFile, IFormFile image1File, IFormFile image2File, IFormFile image3File, IFormFile image4File, IFormFile image5File)
{
    if (ModelState.IsValid)
    {
        // Gọi hàm xử lý và lưu ảnh
        await SaveImage(imageFile, p => product.Image = p);
        await SaveImage(image1File, p => product.Image1 = p);
        await SaveImage(image2File, p => product.Image2 = p);
        await SaveImage(image3File, p => product.Image3 = p);
        await SaveImage(image4File, p => product.Image4 = p);
        await SaveImage(image5File, p => product.Image5 = p);

        _context.Add(product);
        await _context.SaveChangesAsync();
        TempData["SuccessMessage"] = "Sản phẩm đã được thêm thành công.";
        return RedirectToAction(nameof(Index));
    }
    ViewData["categoryId"] = new SelectList(_context.categories, "categoryId", "categoryname", product.categoryId);
    ViewData["typeId"] = new SelectList(_context.types, "typeId", "typename", product.typeId);
    return View(product);
}

0 references
public async Task<IActionResult> Edit(
    int id,
    [Bind("Id,Name,tag,Detail,Price,Banner,Hotupdate,top10,Newproduct,Lastupdate,Image,Image1,Image2,Image3,Image4,Image5,categoryId,typeId")] Product product,
    IFormFile? imageFile, IFormFile? image1File, IFormFile? image2File, IFormFile? image3File, IFormFile? image4File, IFormFile? image5File)
{
    if (id != product.Id)
    {
        return NotFound();
    }

    if (ModelState.IsValid)
    {
        try
        {
            // Lấy thông tin sản phẩm hiện tại từ DB để xử lý xóa ảnh cũ
            var existingProduct = await _context.Products.AsNoTracking().FirstOrDefaultAsync(p => p.Id == id);
            if (existingProduct == null)
            {
                return NotFound();
            }

            // Xử lý từng ảnh một
            await UpdateAndSaveImage(imageFile, product.Image, p => product.Image = p);
            await UpdateAndSaveImage(image1File, product.Image1, p => product.Image1 = p);
            await UpdateAndSaveImage(image2File, product.Image2, p => product.Image2 = p);
            await UpdateAndSaveImage(image3File, product.Image3, p => product.Image3 = p);
            await UpdateAndSaveImage(image4File, product.Image4, p => product.Image4 = p);
            await UpdateAndSaveImage(image5File, product.Image5, p => product.Image5 = p);

            // Cập nhật sản phẩm vào database
            _context.Update(product);
            await _context.SaveChangesAsync();
            TempData["SuccessMessage"] = "Sản phẩm đã được cập nhật thành công.";
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!ProductExists(product.Id))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }
        return RedirectToAction(nameof(Index));
    }
    ViewData["categoryId"] = new SelectList(_context.categories, "categoryId", "categoryname", product.categoryId);
    ViewData["typeId"] = new SelectList(_context.types, "typeId", "typename", product.typeId);
    return View(product);
}
```



```

0 references
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var product = await _context.Products
        .Include(p => p.category)
        .Include(p => p.type)
        .FirstOrDefaultAsync(m => m.Id == id);
    if (product == null)
    {
        return NotFound();
    }

    return View(product);
}

```

```

0 references
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var product = await _context.Products.FindAsync(id);
    if (product != null)
    {
        // Xóa tất cả các file ảnh liên quan
        DeleteImageFile(product.Image);
        DeleteImageFile(product.Image1);
        DeleteImageFile(product.Image2);
        DeleteImageFile(product.Image3);
        DeleteImageFile(product.Image4);
        DeleteImageFile(product.Image5);

        _context.Products.Remove(product);
        await _context.SaveChangesAsync();
        TempData["SuccessMessage"] = "Sản phẩm đã được xóa thành công.";
    }
    return RedirectToAction(nameof(Index));
}

```

```

1 reference
private bool ProductExists(int id)
{
    return _context.Products.Any(e => e.Id == id);
}

```

// Helper method để lưu một file ảnh và cập nhật thuộc tính Image

7 references

```

private async Task SaveImage(IFormFile? file, Action<string?> updateProperty)
{
    if (file != null)
    {
        string wwwRootPath = _hostEnvironment.WebRootPath;
        string uploadsFolder = Path.Combine(wwwRootPath, "images");

        if (!Directory.Exists(uploadsFolder))
        {
            Directory.CreateDirectory(uploadsFolder);
        }

        string fileName = Guid.NewGuid().ToString() + Path.GetExtension(file.FileName);
        string filePath = Path.Combine(uploadsFolder, fileName);

        using (var fileStream = new FileStream(filePath, FileMode.Create))
        {
            await file.CopyToAsync(fileStream);
        }
        updateProperty("/images/" + fileName);
    }
    // Nếu file là null, không làm gì cả, thuộc tính Image sẽ giữ giá trị null hoặc rỗng
}

```

2.6 Triển khai cài đặt

2.6.1 Cấu hình máy phát triển

- Hệ điều hành: Windows 11 64-bit
- CPU: Intel Core i5-11400H
- RAM: 16 GB DDR4
- GPU: GTX 1650 GDDR6
- IDE: Visual Studio 2022 Community, MSSM 2022
- Trình duyệt kiểm thử: Google Chrome, Microsoft Edge

2.6.2 Môi trường phát triển

- Ngôn ngữ: C#
- Kiến trúc: ASP.NET MVC (Model – View – Controller)
- Razor View Engine: để xây dựng giao diện người dùng
- Entity Framework Core (Code First): để làm việc với cơ sở dữ liệu
- Microsoft SQL Server (LocalDB): hệ quản trị cơ sở dữ liệu
- ASP.NET Core Identity: để xử lý đăng nhập, phân quyền, quản lý người dùng
- Bootstrap : xây dựng giao diện responsive, thân thiện
- Font Awesome: sử dụng icon
- CSS & JavaScript : để tạo hiệu ứng, cải thiện trải nghiệm giao diện
- .NET SDK: Phiên bản .NET 8.0

2.6.3 Cài đặt

1. Mở Visual studio 2022 tạo project ASP.Net Core (MVC)
2. Cài bộ thư viện EF và Identity
3. Khởi Tạo Model và Data
4. Cấu hình appsetting.json và program
5. Chạy lệnh add-migration và update-database để tạo CSDL theo CodeFisrt
6. Thực hiện viết các controller xử lý logic và view để hiển thị
7. Nhấn Run/Build để chạy.
8. Kiểm thử test lỗi.

CHƯƠNG 3 KẾT QUẢ CHƯƠNG TRÌNH

3.1 User

3.1.1 Trang chủ

≡

atsumaru

Tìm kiếm

🛒

👤

Omniscient Reader's Viewpoint

action • adventure • Manhwa • drama • fantasy • shounen

Cuối cùng, Omniscient Reader's Viewpoint cũng chủ yếu nói về cách độc giả trôi nổi vào những câu chuyện. Bộ truyện khám phá mối quan hệ giữa tác giả, độc giả và chính câu chuyện. Khoảnh khắc độc giả yêu thích một câu chuyện cũng là lúc câu chuyện đó sống động, thở và có tiềm năng trở nên bất tử...

More info

×

Home

Thể Loại ▾

Loại Truyện ▾

🛒 Giỏ Hàng

♡ Wishlist

🛒 Đơn Hàng Của Bạn

👤 Login/Register

💬 Contact

Hot updates



Lout OF COUNT FAMILY



The Gamer



Material Pea

Top 10



ONE PIECE



BERSERK



JUJUTSU KAISEI

New product



VANGABOND



ONE PUNCH MAN



MY DARLING DRE

📧 Đăng ký nhận tin

Nhập email của bạn:

Đăng ký

GIỚI THIỆU

Atsumaru – Nền tảng truyện tranh & thương mại điện tử dành cho fan manga. Đọc truyện, mua sắm, kết nối đam mê!

 ĐẢM BẢO
BỘ CÔNG THƯƠNG

CHÍNH SÁCH

- Liên hệ
- Hình thức thanh toán
- Điều khoản dịch vụ
- Chính sách vận chuyển
- Chính sách đổi trả
- Chính sách bảo mật thông tin

THÔNG TIN LIÊN HỆ

📍 27 Tôn Thất Tùng-Phường 8-TP Đà Lạt

☎ 0123456789

✉ atsumaru@gmail.com

{ 52 }

3.1.2 Chi tiết sản phẩm



130,000đ

Thêm vào giỏ

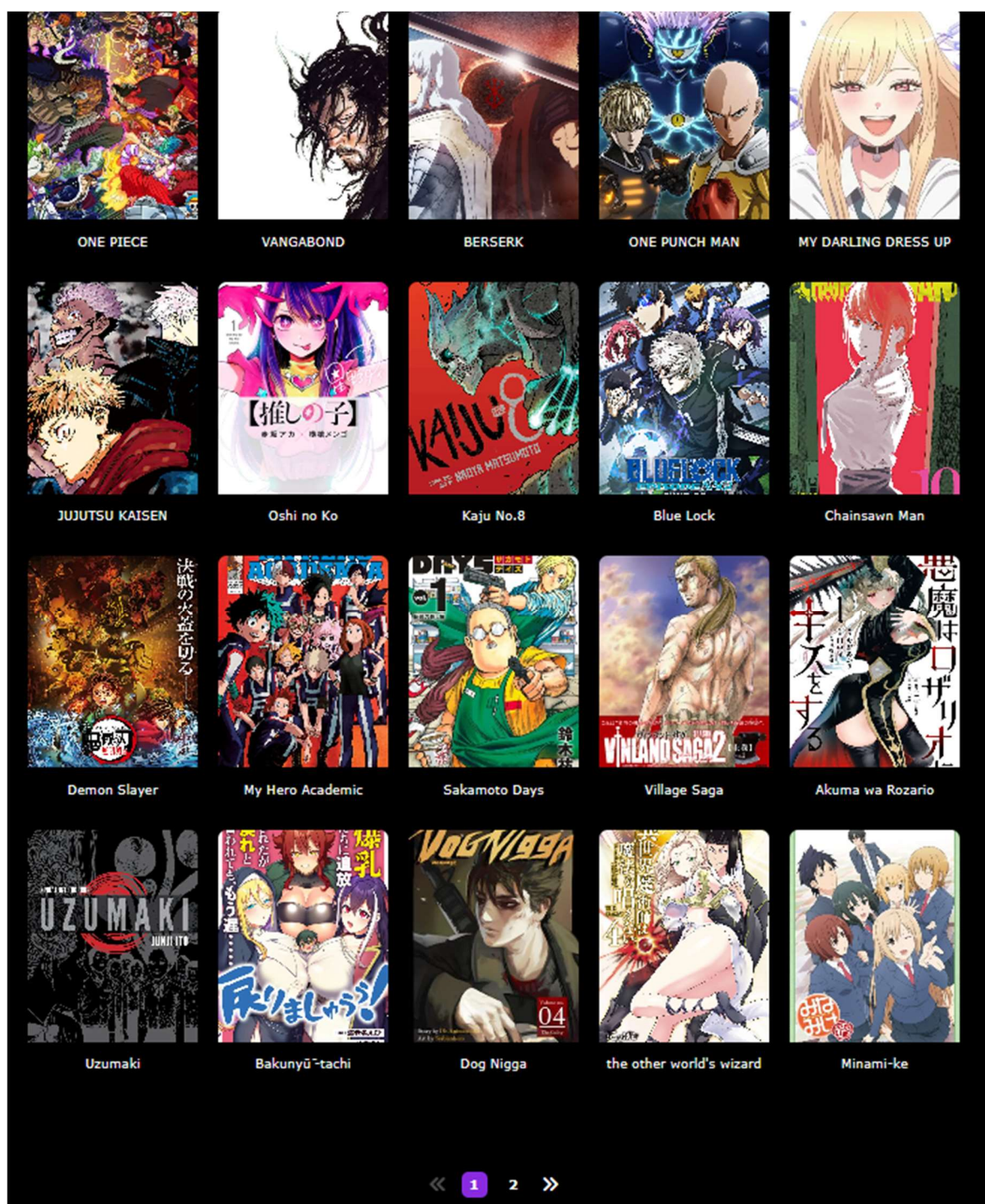
sports • psychological • drama • competition • manga • bóng đá • shounen

Blue Lock" xoay quanh một dự án quốc gia nhằm tạo ra tiền đạo xuất sắc nhất cho đội tuyển Nhật Bản sau thất bại thảm hại tại World Cup. 300 tiền đạo trẻ từ khắp cả nước được tập hợp vào một cơ sở huấn luyện đặc biệt tên là "Blue Lock", nơi chỉ người giỏi nhất mới được sống sót, kẻ thua sẽ bị loại khỏi giấc mơ trở thành tuyển thủ quốc gia mãi mãi. Nhân vật chính Yoichi Isagi – một cầu thủ đầy tiềm năng nhưng thiếu bản lĩnh – bước vào hành trình tìm kiếm "cái tôi ích kỷ" để trở thành số 1. Truyện kịch tính, đậm chất tâm lý và phá cách so với những manga thể thao truyền thống.

Preview



3.1.3 Danh mục/Danh sách yêu thích



3.1.4 Đăng nhập/Đăng ký

Đăng nhập

Tên đăng nhập



Mật khẩu



☐ ghi nhớ đăng nhập?

[Quên mật khẩu?](#)

Đăng nhập

Bạn chưa có tài khoản? [Đăng ký ngay!](#)

[Quay lại trang chủ](#)

Đăng ký

Email



Mật khẩu



Nhập lại mật khẩu

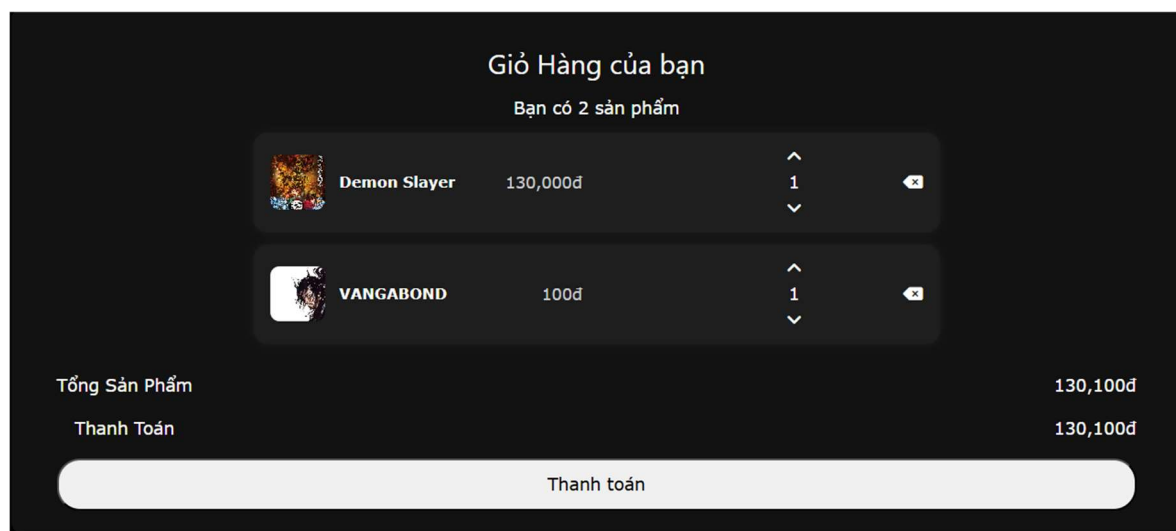


Bạn đã có tài khoản? [Đăng nhập ngay!](#)

Đăng ký

[Quay lại trang chủ](#)

3.1.5 Giỏ hàng



3.1.6 Thanh toán

Thông tin giao hàng

Phương thức vận chuyển

☒ Giao hàng tận nơi

☐ Nhận tại cửa hàng

☒ Giao hàng thường (3-4 ngày)

☐ Giao hàng nhanh (1-2 ngày)

☐ Giao hoá tốc

Phương thức thanh toán

☒ Thanh toán khi nhận hàng (COD)

☐ Chuyển khoản ngân hàng

☐ Ví Momo

Xác nhận

3.1.7 Lịch sử đơn hàng

Lịch sử đơn hàng của bạn						
STT	Thời gian đặt hàng	Chi tiết sản phẩm	Địa chỉ giao hàng	Tổng cộng	Trạng thái	
1	18-06-2025 17:45:03	 1 x VANGABOND (100.00 VNĐ)	dswcxdws	100đ	Đang giao	
2	16-06-2025 14:50:18	 1 x Lout OF COUNT FAMILY (100.00 VNĐ)	scsacsacsa	100đ	Đã Hủy	
3	16-06-2025 14:46:55	 1 x Lout OF COUNT FAMILY (100.00 VNĐ)	scsacsacsa	200đ	Đã Giao	
		 1 x BERSERK (100.00 VNĐ)				
4	16-06-2025 14:26:33	 1 x Lout OF COUNT FAMILY (100.00 VNĐ)	scsacsacsa	100đ	Đã Hủy	
5	16-06-2025 14:19:59	 1 x Lout OF COUNT FAMILY (100.00 VNĐ)	scsacsacsa	100đ	Đã Hủy	
6	16-06-2025 14:13:41	 1 x Lout OF COUNT FAMILY (100.00 VNĐ)	scsacsacsa	100đ	Đang xử lý	
7	16-06-2025 14:12:17	 1 x Lout OF COUNT FAMILY (100.00 VNĐ)	scsacsacsa	100đ	Đang xử lý	
8	16-06-2025 14:11:58	 1 x Lout OF COUNT FAMILY (100.00 VNĐ)	scsacsacsa	100đ	Đang xử lý	

3.1.8 Liên Hệ

Get in Touch

Chúng Tôi Sẽ Liên Hệ Sớm Nhất có Thể

admin@atsumaru.com

Message

Gửi tin nhắn

Email của bạn (**admin@atsumaru.com**) sẽ được sử dụng để liên hệ.

[🕒 Xem lịch sử tin nhắn của bạn](#)

3.1.9 Profile

Thông Tin Cá Nhân



ID Người dùng: edc801a7-2dcc-4287-be53-79f1d9254da4

Tên đăng nhập: admin@atsumaru.com

Email: admin@atsumaru.com

[Đổi mật khẩu](#) [Phản Hồi](#)

Sản phẩm yêu thích của bạn



Lout OF COUNT FAMI...



MY DARLING DRESS ...



VANGABOND

3.1.10 Đổi mật khẩu/Phản hồi

Đổi mật khẩu

Mật khẩu cũ

Mật khẩu mới

Xác nhận mật khẩu mới

Đổi mật khẩu

Lịch sử tin nhắn của bạn

Các tin nhắn của bạn (1):

Gửi lúc: 16-06-2025 13:39

Nội dung tin nhắn: hello

Phản hồi từ Admin:

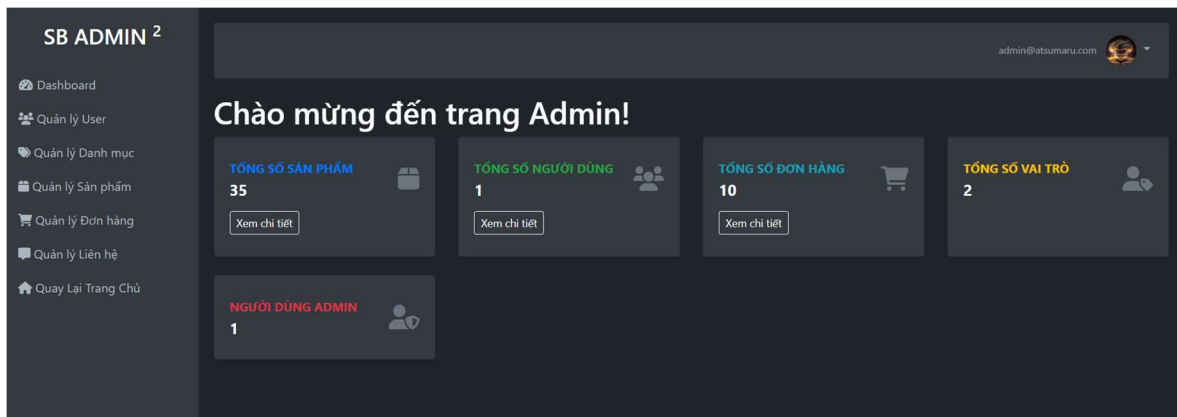
im mọc mồm

Đã phản hồi vào: 16-06-2025 13:40

Gửi tin nhắn mới

3.2 Admin

3.2.1 Dashboard



3.2.2 Quản Lý User

Quản lý Người dùng		
UserName	Email	PhoneNumber
admin@atsumaru.com	admin@atsumaru.com	<button>Sửa</button> <button>Chi tiết</button> <button>Xóa</button>

3.2.3 Quản lý Danh mục

Quản lý Category		
<button>Thêm Category mới</button>		
Chọn loại: <button>Danh mục</button> <button>Loại truyện</button>		
Tên Category		
Shounen	<button>Chỉnh sửa</button> <button>Chi tiết</button> <button>Xóa</button>	
Seinen	<button>Chỉnh sửa</button> <button>Chi tiết</button> <button>Xóa</button>	
Kodomo	<button>Chỉnh sửa</button> <button>Chi tiết</button> <button>Xóa</button>	
Shoujo	<button>Chỉnh sửa</button> <button>Chi tiết</button> <button>Xóa</button>	

3.2.4 Quản lý sản phẩm

Quản lý Sản phẩm										
Thêm sản phẩm mới Tìm kiếm sản phẩm... Tìm kiếm Xóa tìm kiếm										
STT	Name	Image	Price	Danh mục	Loại truyện	Hot Update	Top 10	New Product	Last Update	Banner
1	ONE PIECE		100 VNĐ	Shounen	Manga	✗	✓	✗	✓	✓
		Sửa Xóa Chi tiết								
2	SOLO LEVELING		100 VNĐ	Shounen	Manhwa	✗	✗	✗	✗	✓
		Sửa Xóa Chi tiết								
3	VANGABOND		100 VNĐ	Seinen	Manga	✗	✗	✓	✗	✓
		Sửa Xóa Chi tiết								
4	BERSERK		100 VNĐ	Seinen	Manga	✗	✓	✗	✓	✓
		Sửa Xóa Chi tiết								
5	THE DEVIL BUTLER		100 VNĐ	Shoujo	Manhua	✗	✗	✗	✗	✓
		Sửa Xóa Chi tiết								
6	Omniscient Reader's Viewpoint		100 VNĐ	Shoujo	Manhua	✗	✗	✗	✗	✓
		Sửa Xóa Chi tiết								

3.2.5 Quản Lý đơn hàng

Name	Phone	DeliveryPlaced	DeliveryTotal	Status		
Nguyễn Ngọc Vỹ	+840784056534	18-06-2025 17:45	100.00đ	Đang giao	Cập nhật trạng thái	Xem chi tiết
Nguyễn Ngọc Vỹ	0784056534	16-06-2025 14:50	100.00đ	Đã Hủy	Cập nhật trạng thái	Xem chi tiết
Nguyễn Ngọc Vỹ	0784056534	16-06-2025 14:46	200.00đ	Đã Giao	Cập nhật trạng thái	Xem chi tiết
Nguyễn Ngọc Vỹ	0784056534	16-06-2025 14:26	100.00đ	Đã Hủy	Cập nhật trạng thái	Xem chi tiết
Nguyễn Ngọc Vỹ	0784056534	16-06-2025 14:19	100.00đ	Đã Hủy	Cập nhật trạng thái	Xem chi tiết
admin@atsumaru.com	+840784056534	16-06-2025 14:13	100.00đ	Đang xử lý	Cập nhật trạng thái	Xem chi tiết
admin@atsumaru.com	0784056534	16-06-2025 14:12	100.00đ	Đang xử lý	Cập nhật trạng thái	Xem chi tiết
admin@atsumaru.com	0784056534	16-06-2025 14:11	100.00đ	Đang xử lý	Cập nhật trạng thái	Xem chi tiết
Nguyễn Ngọc Vỹ	+840784056534	12-06-2025 22:45	100.00đ	Đang giao	Cập nhật trạng thái	Xem chi tiết
Cô giáo Thảo	0784056534	12-06-2025 22:45	100000.00đ	Đã Hủy	Cập nhật trạng thái	Xem chi tiết

3.2.6 Quản lý liên hệ

Quản lý Liên hệ					
Name	Email	Message	SentDate	AdminReply	
admin@atsumaru.com	admin@atsumaru.com	hello	16-Jun-25 1:39:51 PM	Đã phản hồi	Xem chi tiết & Phản hồi Xóa
Nguyễn Ngọc Vỹ	atushidaiki@gmail.com	ừqew	15-Jun-25 12:27:18 PM	Đã phản hồi	Xem chi tiết & Phản hồi Xóa
SOLO LEVELING	atushidaiki@gmail.com	dumamay	12-Jun-25 10:44:38 PM	Đã phản hồi	Xem chi tiết & Phản hồi Xóa
haha	atushidaiki@gmail.com	nhu db	12-Jun-25 10:09:21 PM	Đã phản hồi	Xem chi tiết & Phản hồi Xóa

3.3 Tổng quan

STT	Chức Năng	Tiến Độ
1	Đăng Ký	Hoàn thành
2	Đăng nhập	Hoàn thành
3	Thêm sản phẩm vào giỏ hàng	Hoàn thành
4	Thêm sản phẩm vào danh sách yêu thích	Hoàn thành
5	Tìm kiếm	Hoàn thành
6	Xem chi tiết sản phẩm	Hoàn thành
7	Cập nhật giỏ hàng	Hoàn thành
8	Đặt hàng	Hoàn thành
9	Thêm sản phẩm/Danh mục	Hoàn thành
10	Sửa sản phẩm/Danh mục	Hoàn thành
11	Cập nhật trạng thái đơn hàng	Hoàn thành
12	Xóa sản phẩm /Danh mục	Hoàn thành
13	Quản lý đơn hàng	Hoàn thành
14	Thanh toán trực tuyến	Đang phát triển
15	Liên hệ/phản hồi	Hoàn thành
16	Đổi mật khẩu cá nhân	Hoàn thành
17	Chỉnh sửa thông tin cá nhân	Đang phát triển

CHƯƠNG 4 : KIẾT LUẬN

4.1 Tổng kết kiến thức đạt được

Qua quá trình thực hiện đồ án, mình đã củng cố và vận dụng nhiều kiến thức lý thuyết và thực tiễn, bao gồm:

- Hiểu rõ và áp dụng mô hình MVC (Model – View – Controller) trong xây dựng ứng dụng web, giúp tổ chức mã nguồn rõ ràng, dễ bảo trì.
- Làm việc với ASP.NET Core MVC, sử dụng Razor View Engine để xây dựng giao diện động, hỗ trợ hiển thị dữ liệu hiệu quả.
- Áp dụng Entity Framework Core theo hướng Code First để thiết kế cơ sở dữ liệu, định nghĩa các quan hệ giữa bảng và thực hiện các thao tác CRUD linh hoạt.
- Tích hợp ASP.NET Core Identity để xử lý đăng ký, đăng nhập, phân quyền người dùng một cách an toàn và mở rộng.
- Sử dụng Bootstrap, CSS/JS, AJAX để tạo giao diện thân thiện, hiện đại, hỗ trợ tương tác không cần tải lại trang, giúp nâng cao trải nghiệm người dùng.
- Rèn luyện kỹ năng làm việc với Visual Studio, SQL Server và tư duy thiết kế hệ thống web thực tế. Đồng thời nâng cao khả năng tự học, giải quyết lỗi phát sinh trong quá trình triển khai và tối ưu hiệu suất ứng dụng.

4.2 Điểm tồn tại

Mặc dù đã hoàn thành hầu hết các chức năng đề ra, nhưng đồ án vẫn còn một số hạn chế như:

- Tính bảo mật cơ bản có nhưng chưa được kiểm thử kỹ lưỡng và chưa áp dụng các biện pháp mã hóa nâng cao.
- Chưa có API xác thực thanh toán để kết nối với các cổng thanh toán thực tế
- Chưa có hệ thống chatbox
- Chưa tích hợp chức năng gửi email thông báo đơn hàng, xác nhận đăng ký hay quên mật khẩu.

- Giao diện chưa thực sự tối ưu cho mobile.
- Chưa có hệ thống chức năng theo dõi vị trí đơn hàng, ví dụ như trạng thái vận chuyển hoặc khoảng cách ước tính.
- Chưa có tính năng thống kê doanh thu cho admin.
- Chưa có hệ thống nhập kho, số lượng sản phẩm, tồn kho.

4.3 Hướng phát triển dự án

Trong tương lai, để nâng cấp và hoàn thiện hệ thống, em định hướng mở rộng các tính năng sau:

- Tích hợp API cho phép thanh toán nhiều ngân hàng.
- Tự động gửi email thông báo cho đơn hàng.
- Tích hợp tính năng quên mật khẩu.
- Tối ưu giao diện cho mobile
- Cải thiện hiệu suất và bảo mật
- Tích hợp Google map API để cho khách hàng dễ theo dõi đơn hàng
- Thêm số lượng tồn kho, nhập kho.
- Trang quản trị viên để quản lý sản phẩm (truyện), đơn hàng, người dùng, kiểm tra số liệu thống kê, theo dõi hiệu suất bán hàng theo từng giai đoạn.
- Bên cạnh đó, hệ thống còn được thiết kế với khả năng mở rộng trong tương lai, đánh giá truyện, bình luận hoặc livestream giới thiệu truyện mới nhằm tăng tương tác cộng đồng.

4.4 Tài liệu tham khảo:

- [1] Microsoft Docs – ASP.NET Core MVC – Microsoft – Online – 2023
<https://learn.microsoft.com/en-us/aspnet/core/mvc>
- [2] Entity Framework Core Documentation – Microsoft – Online – 2023
<https://learn.microsoft.com/en-us/ef/core/>
- [3] ASP.NET Core Identity Overview – Microsoft – Online – 2023
<https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity>

- [4] Xây dựng website với ASP.NET Core MVC – Ths. Nguyễn Đức Tấn – 2025
Đại học Yersin (Bài giảng nội bộ)
- [6] ASP.NET Core toàn tập – ThS. Nguyễn Đức Tấn – 2025 – Trường Đại Học
Yersin (Giáo trình nội bộ)
- [7] ASP.NET Core MVC Full Project Tutorial – IAmTimCorey (YouTube) – 2021
<https://www.youtube.com/watch?v=FbfkYjIg8Fw>
- [8] Learn ASP.NET Core MVC in 10 Hours – FreeCodeCamp (YouTube) – 2023
<https://www.youtube.com/watch?v=C5cnZ-gZy2I>
- [9] ASP.NET Core MVC Tutorial for Beginners – Kudvenkat (YouTube) – 2022
<https://www.youtube.com/watch?v=U3ESZZ7HG84>